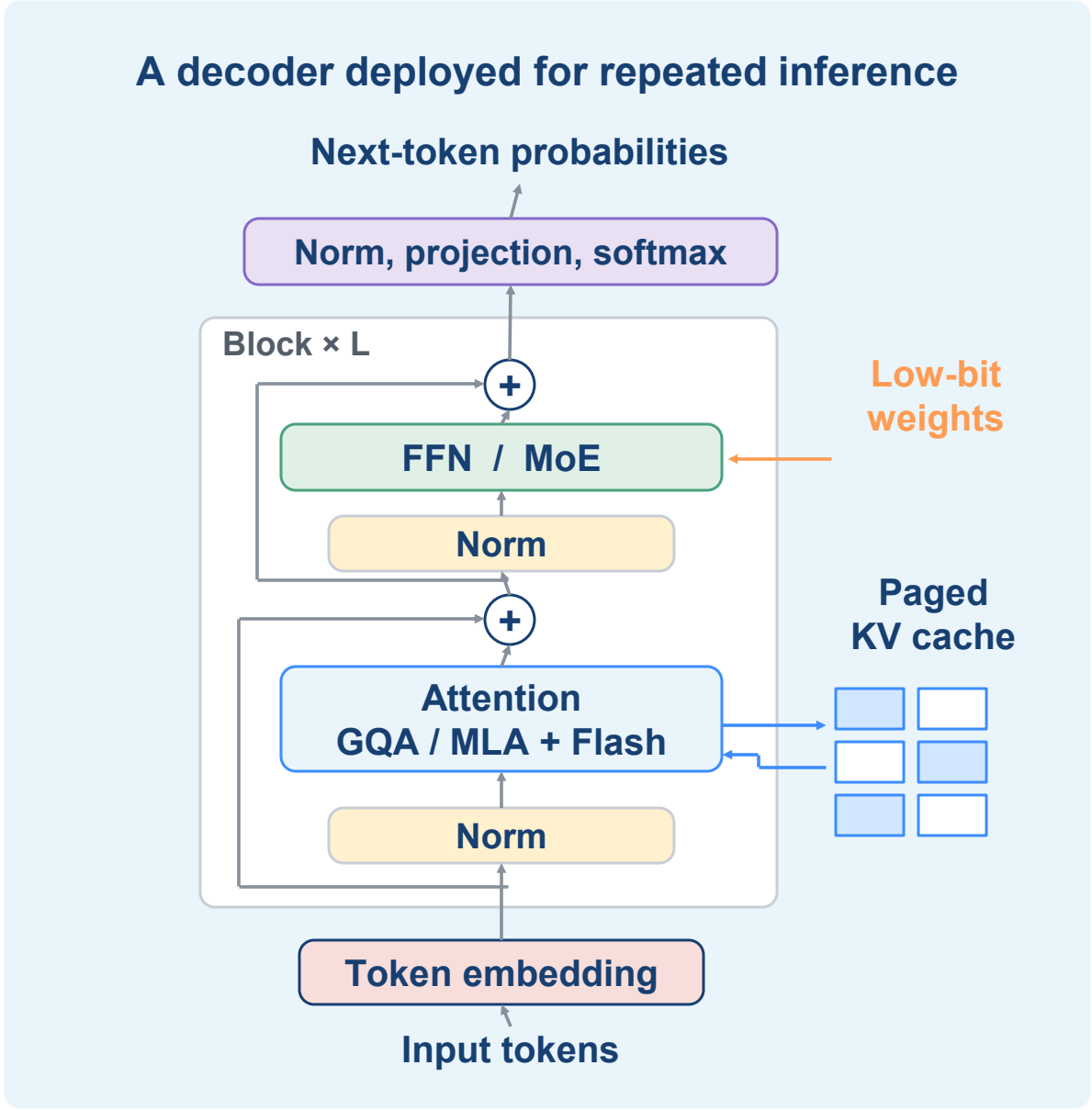
COMP7608B · LECTURE 2

Efficient LLM Inference

Architecture and high-performance serving

Wenbo Sun

School of Computing and Data Science
HKU



What inference costs

The company pays

GPU count × rental rate × billed hours

Memory determines what fits.
Throughput determines how much traffic each GPU can serve.

The user waits



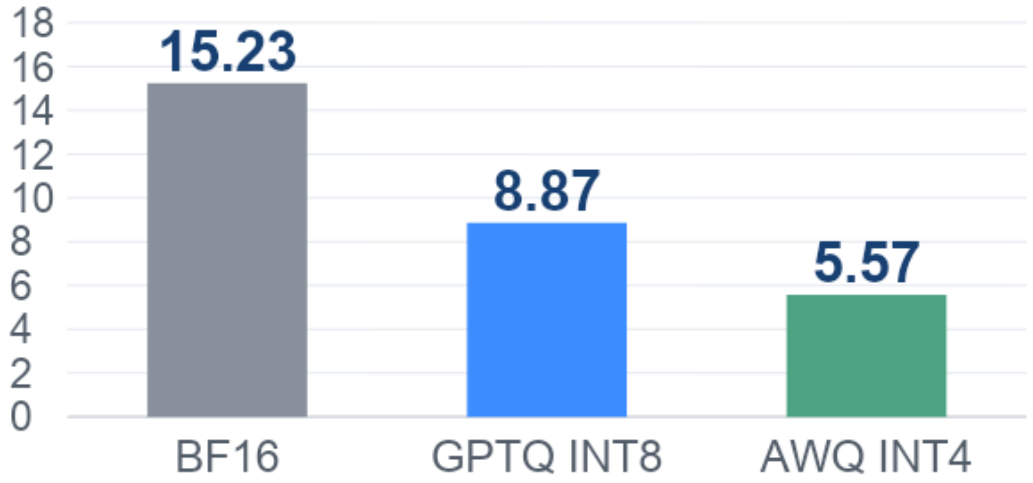
Time to first token
Time between generated tokens
Time to the complete answer

A deployable service must meet a quality target and a latency target at an affordable cost.

Quantization changes the memory requirement

Stored tensor payload (GB), including scales and higher-precision tensors.

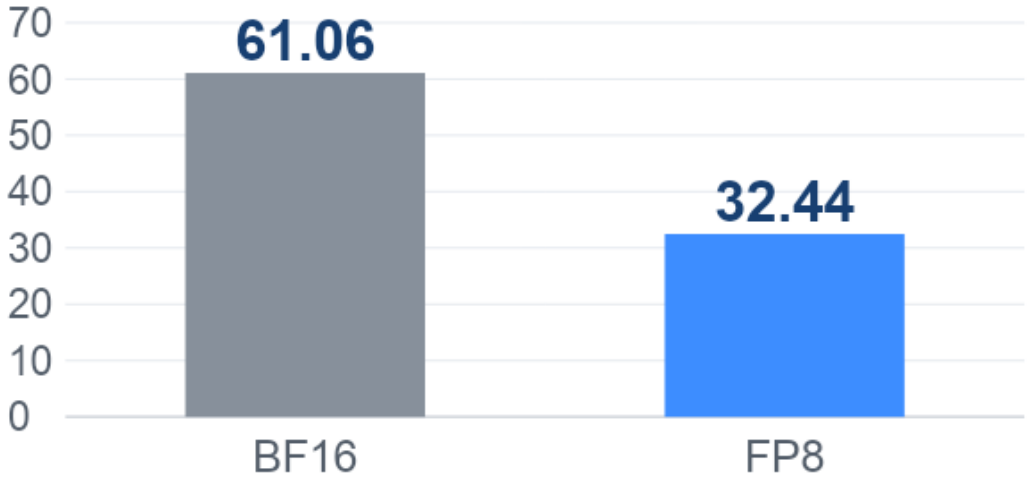
Dense · Qwen2-7B-Instruct



INT8 retains 58.2% of BF16 tensor bytes.

MoE · Qwen3-30B-A3B

30.5B total, 3.3B active per token



FP8 retains 53.1% of BF16 tensor bytes.

MoE still stores all experts. Lower precision reduces this stored weight state.

GB = 10^9 bytes. Tensor payload from checkpoint indices, including scale tensors. Excludes file headers, KV cache and runtime buffers. Separate chart scales.

Quantization reduces token-generation latency

Llama-7B on one A100 80GB, batch size 1, GPU power limited to 330 W



47%

less decoding time with INT4

All three paths use compilation. INT4 also uses calibration and specialized kernels.

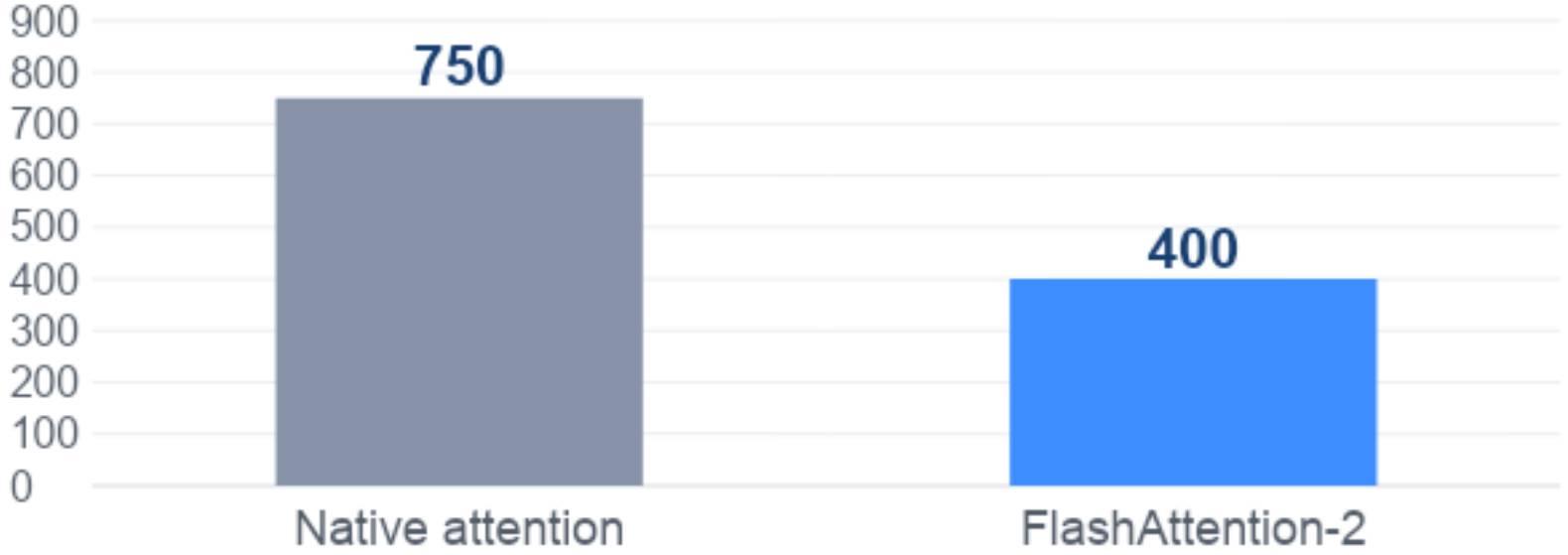
Lower precision can reduce weight-transfer time when the runtime has efficient kernels.

PyTorch, GPT Fast (2023)

FlashAttention reduces prompt latency

Llama-2-7B on one NVIDIA A100, batch size 1, 4,096 tokens, no padding

One full-model forward pass (ms), approximate figure readings



~47%

less time per forward pass

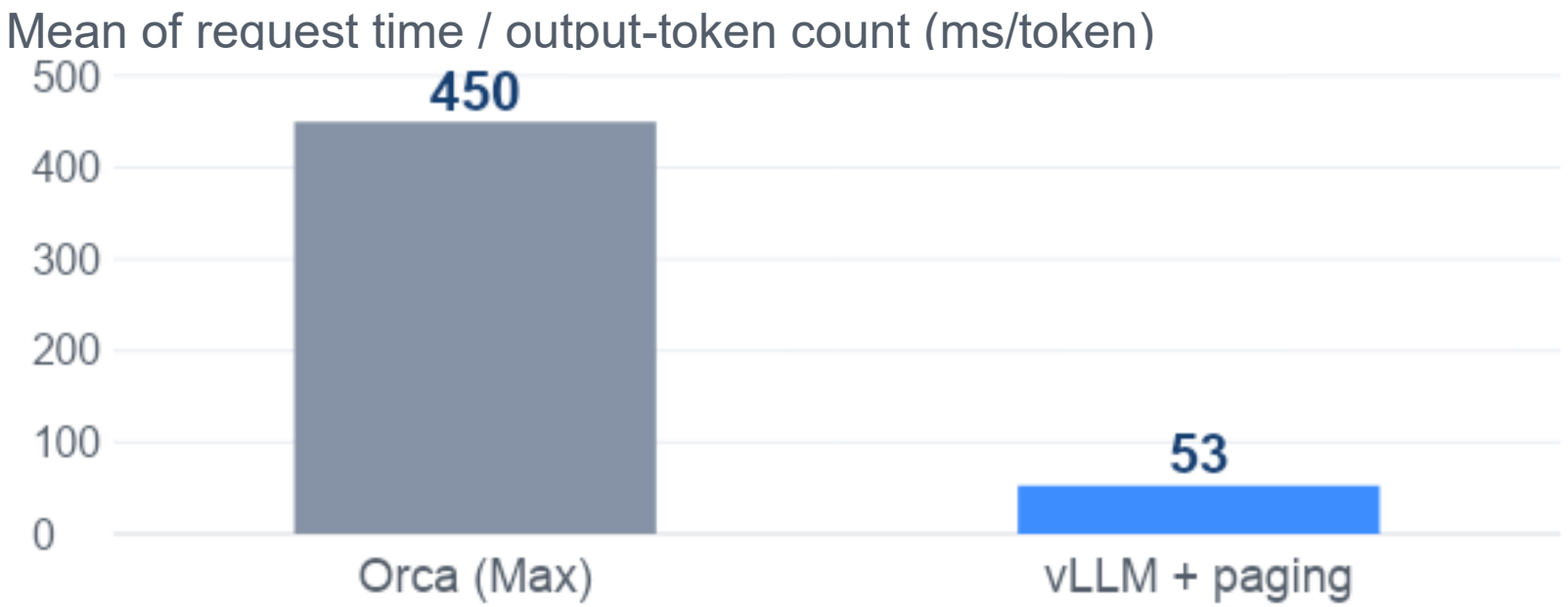
This measures prompt-style computation. It does not include a complete generated answer.

FlashAttention reduces attention memory traffic while preserving the attention calculation.

Published Llama-2-7B timing figure

PagedAttention helps latency under traffic

OPT-13B on one A100 40GB, ShareGPT chatbot trace, 0.55 requests/s



~88%

lower normalized request latency

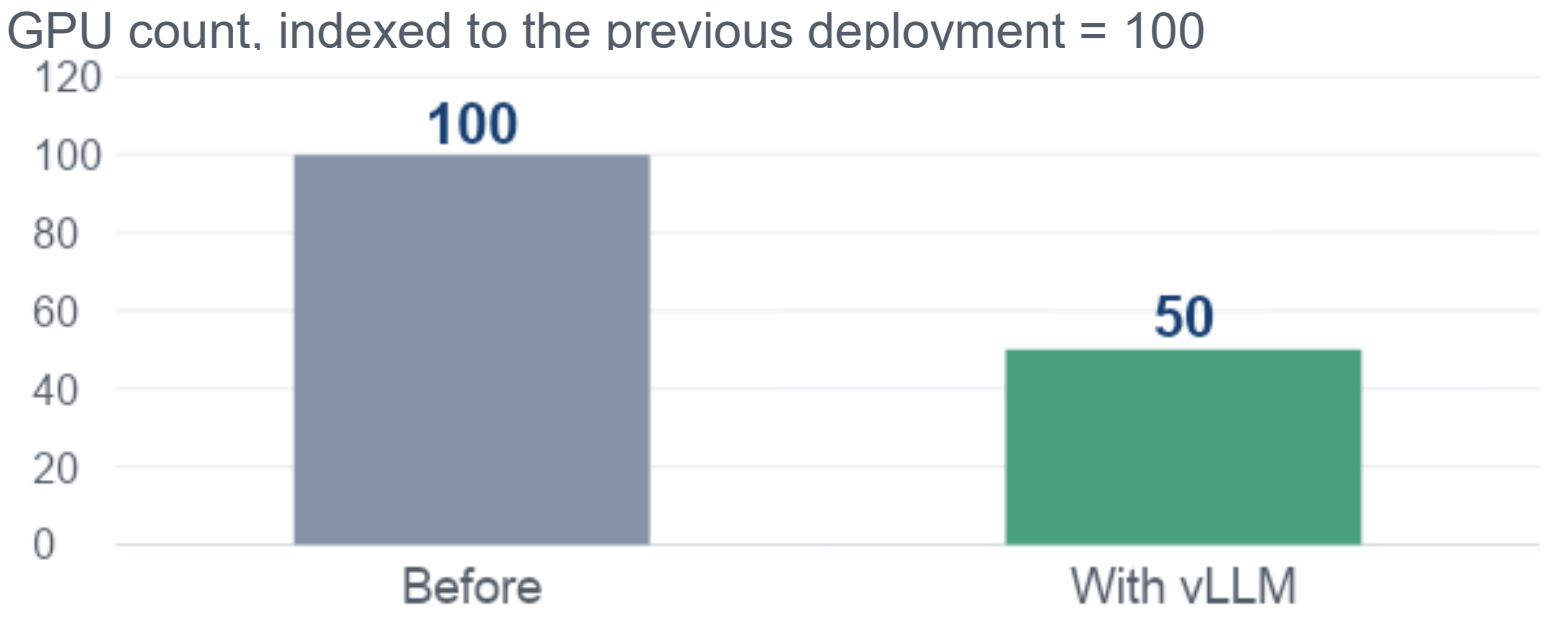
Includes queueing and prompt work. Approximate values from the published curve.

Paging allows more live requests to fit. The system's batching and scheduling also contribute.

Kwon et al., PagedAttention (2023), Fig. 17

A public service used half as many GPUs

LMSYS adopted FastChat-vLLM for Vicuna and Chatbot Arena in 2023



30,000

requests per day on average

60,000

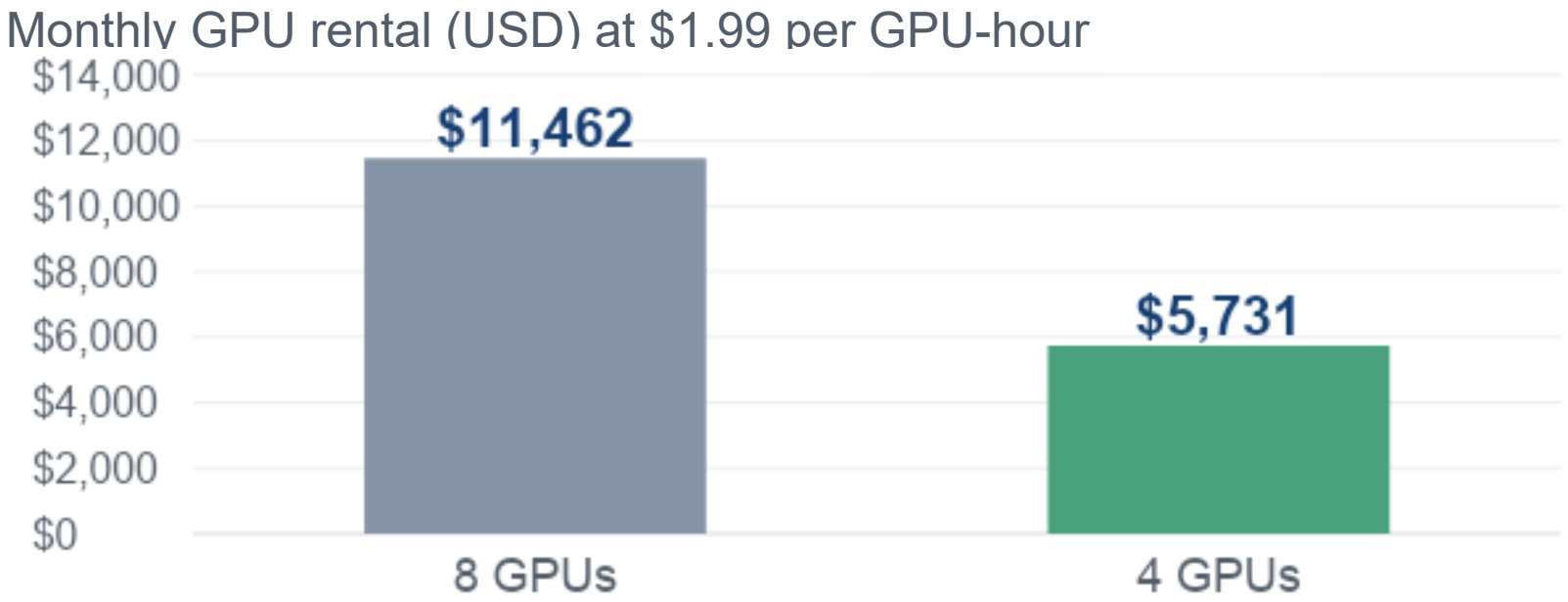
requests on peak days

The team reported a 50% GPU reduction. Efficient serving changed the hardware requirement.

vLLM, FastChat / LMSYS deployment (2023)

What a smaller public-service fleet saves

Planning example: 8 to 4 A100 40GB GPUs, each billed for 720 hours/month



$5,731

saved per month

$68,774 per year

Assumption: the optimized fleet handles the same workload and meets the same service target.

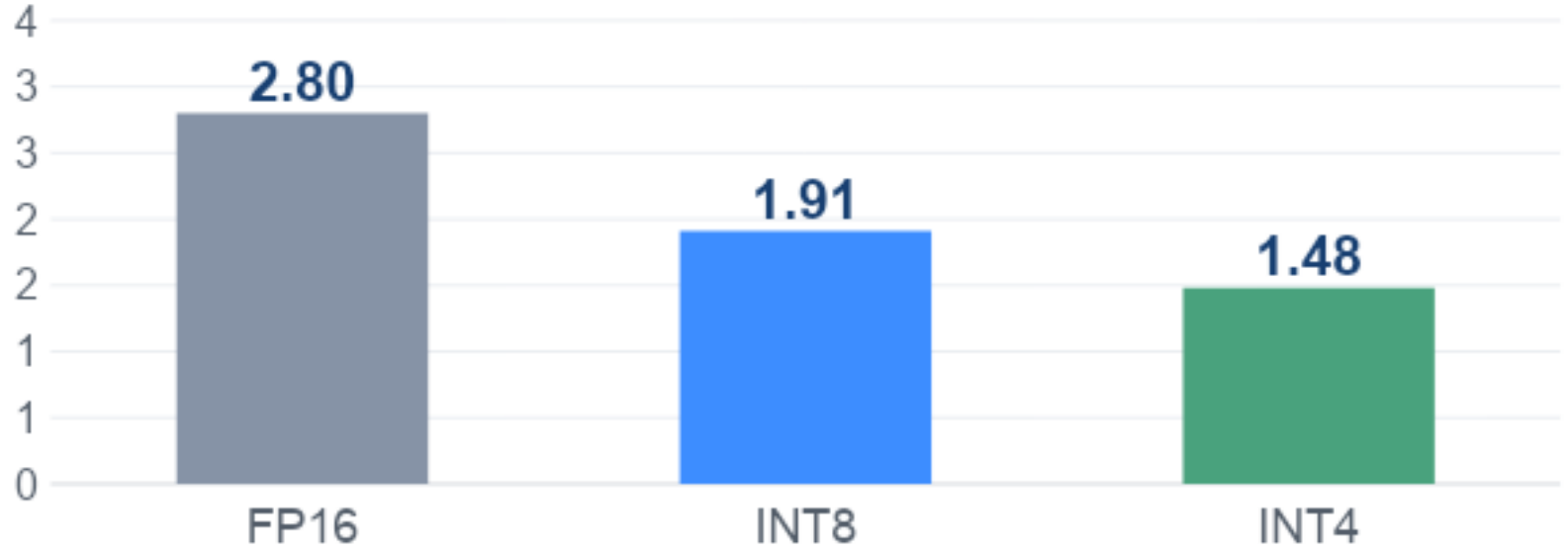
8 × $1.99 × 720 = $11,462/month. Halving billed capacity halves this rental cost.

Lambda pricing, checked 10 Sep 2026 / vLLM, FastChat / LMSYS deployment (2023)

An internal assistant can return answers sooner

Example workload: 1,000 employees × 20 answers/day × 22 working days

Decoding time for a 300-token answer (s), derived from slide 4



1.32 s

less waiting per answer with INT4

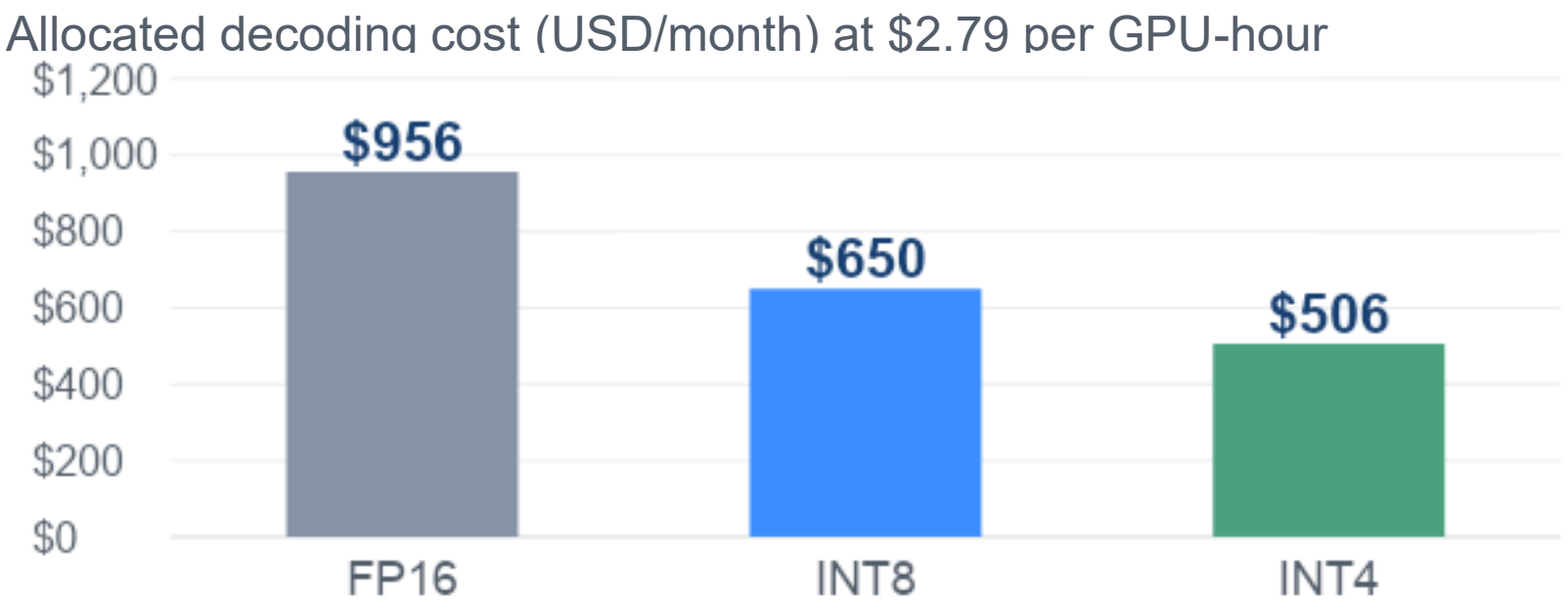
440,000 answers/month = 132 million output tokens.

These are decoding-time estimates. Queueing and prompt processing add to the complete response time.

PyTorch, GPT Fast (2023)

Internal usage also accumulates compute cost

132 million output tokens/month, using the same measured generation rates



$450

lower monthly GPU-time allocation

GPU-hours = output tokens / tokens per second / 3,600. Prefill and idle time are excluded.

Lower billed GPU-hours produce cash savings. Idle reserved capacity still costs money.

PyTorch, GPT Fast (2023) / Lambda pricing, checked 10 Sep 2026

The engineering questions for Lecture 2

Part	Constraint	Question
Quantization	Weight bytes and bandwidth	How much precision can the task afford?
GQA, MQA	KV state per active request	How much context can we keep?
Flash + PagedAttention	Attention traffic and allocation	How should the engine move and store data?
MoE architecture	Active work and expert storage	Which parameters run for each token?
Serving systems	Concurrency and queueing	Which configuration meets the latency target?

Success means acceptable task quality, low tail latency and enough sustained throughput for the workload.

FP32, FP16 and BF16 bit layouts

Sign sets polarity. Exponent sets range. Fraction sets precision.

FP32



32 bits = 1 sign + 8 exponent + 23 fraction

FP16



16 bits = 1 sign + 5 exponent + 10 fraction

More fraction bits than BF16

BF16



16 bits = 1 sign + 8 exponent + 7 fraction

Same exponent width as FP32

Both 16-bit formats halve FP32 storage. BF16 keeps a wider range, while FP16 keeps finer precision.

NVIDIA, reduced-precision formats

How computers store floating-point numbers

The same value in FP32 and BF16 x = 6.5078125

$$x = (-1)^s \left(1 + \frac{F}{2^m}\right) 2^{E-B}$$

s = sign, E = stored exponent, B = bias, F = fraction as an integer, m = fraction bits

Format	Sign	Exponent	Fraction bits	Stored value
FP32	0	10000001	10100000100000000000000	6.5078125
BF16	0	10000001	1010000	6.5

FP32: m = 23, F = 5,259,264 BF16: m = 7, F = 80 Both: E = 129, B = 127

$$x_{\mathrm{BF16}} = (1 + 80/128)\, 2^{129-127} = 6.5$$

BF16 stores 6.5. Absolute error = 0.0078125, or about 0.12% of the original value.

Round to nearest. FP16 represents this particular value exactly. Formula applies to normal finite values. Source: NVIDIA, Floating Point and IEEE 754.

Quantization formula and parameter selection

$$q = \mathrm{clip}(\mathrm{round}(x / s) + z, q_{min}, q_{max}) \qquad \hat{x} = s \times (q - z)$$

$s > 0$ is the step size. z is the integer code for zero. $\hat{x}$ approximates x.

1 Choose bit width and integer range

Unsigned 4-bit uses [0, 15].
Symmetric signed 4-bit often uses [−7, 7], leaving the −8 storage code unused.

2 Select a real range [a, b] including zero

Use weight statistics or representative activation samples. Start with min/max. Percentile or error-based clipping may help.

Asymmetric

$$s = (b - a) / (q_{max} - q_{min})$$
$$z = \mathrm{clip}(\mathrm{round}(q_{min} - a / s), q_{min}, q_{max})$$

Symmetric

$A = \max(|a|, |b|) \quad s = A / 7 \quad z = 0$ (4-bit example)

Two 4-bit quantization examples

Symmetric weights

Observed range $[-3.5, 3.5]$
q range $[-7, 7]$

$s = 3.5 / 7 = 0.5$
$z = 0$

$x = 1.3$
$q = \text{round}(1.3 / 0.5) = 3$
Stored bits 0011
$\hat{x} = 0.5 \times 3 = 1.5$

Asymmetric range

Observed range $[-1.0, 2.0]$
q range $[0, 15]$

$s = 3 / 15 = 0.2$
$z = \text{round}(0 - (-1) / 0.2) = 5$

$x = 0.74$
$q = \text{round}(0.74 / 0.2) + 5 = 9$
Stored bits 1001
$\hat{x} = 0.2 \times (9 - 5) = 0.8$

Absolute errors are 0.20 and 0.06. Reconstructing either value requires its scale and zero-point.

Outliers make one scale too coarse

One symmetric 4-bit scale for all five values

A = 7.0 s = 7.0 / 7 = 1.0 z = 0

Original x	−0.4	−0.2	0.1	0.3	7.0
Stored q	0	0	0	0	7
Reconstructed $\hat{x}$	0.0	0.0	0.0	0.0	7.0

The outlier preserves its value, while four ordinary weights collapse to zero. Clipping trades one error for another.

Smaller groups can use different scales

Symmetric INT4, round to nearest even

Group	Original values	Scale	Stored q
A	[−0.4, −0.2, 0.1, 0.3]	0.4 / 7	[−7, −4, 2, 5]
B	[7.0, 3.5, 0.0, −7.0]	1.0	[7, 4, 0, −7]

For 128 weights, 4-bit values occupy 64 bytes.
One FP16 scale adds 2 bytes, or 3.1%, before zero-points and other metadata.

Smaller groups adapt to local ranges, but require more scales and support from the inference kernel.

Error accumulation and mixed precision

Start at 2048. Add 1 four times. The exact answer is 2052.

Accumulator	After +1	After +1	After +1	After +1	Final error
FP32 reference	2049	2050	2051	2052	0
FP16 each step	2048	2048	2048	2048	4
FP16 inputs, FP32 sum	2049	2050	2051	2052	0

At 2048, FP16 spacing is 2. Each 2048 + 1 rounds back to 2048.

Use higher precision for sums, normalization and softmax to limit further error.

Upcasting cannot recover bits already lost. Storage and computation can use different precisions.

Quantized models already mix formats

Qwen2-7B-Instruct-AWQ — INT4 linear weights with FP16 activations.

Qwen3-30B-A3B-FP8 — FP8 weights, with routers and LM head kept in BF16.

Toy sequential sum, round-to-nearest-even. Actual accumulation depends on the kernel. Sources: NVIDIA accuracy guidance and the named model configs.

Post-training quantization and QAT

Post-training quantization (PTQ)

A trained checkpoint is already available. Quantize it using weight statistics and, for methods such as GPTQ or AWQ, representative calibration inputs.

Checkpoint → **Calibration + conversion** → **Quantized checkpoint**

Quantization-aware training (QAT)

Simulate quantize + dequantize in the forward pass, then update weights with gradients. Export real low-bit weights after training or fine-tuning.

Rounding uses an approximate backward gradient, often a straight-through estimator.

PTQ is a low-cost first deployment candidate. QAT adds training cost to help recover quality.

PyTorch, Quantization-Aware Training for LLMs / Frantar et al., GPTQ / Lin et al., AWQ

Common quantization methods

Method	Typical use	What it changes
RTN	PTQ baseline	Round each value to a chosen grid.
GPTQ	PTQ, often W4A16	Use calibration inputs and curvature information to reduce layer-output error.
AWQ	PTQ, often W4A16	Use activation statistics to rescale important weight channels.
SmoothQuant	PTQ, W8A8	Rescale activation outliers into weights so both can use INT8.
QAT	Training / fine-tuning	Learn with simulated quantization error, then convert for inference.

GPTQ and AWQ commonly appear in model cards.
FP8 / FP4 are formats. PTQ / QAT describe how a quantized model is produced.

Frantar et al., GPTQ / Lin et al., AWQ / Xiao et al., SmoothQuant / PyTorch, Quantization-Aware Training for LLMs

Quantization, speed, storage and accuracy

Qwen2-7B-Instruct, one A100 80GB, vLLM 0.4.2
Batch 1, 1 input token, 2,048 output tokens

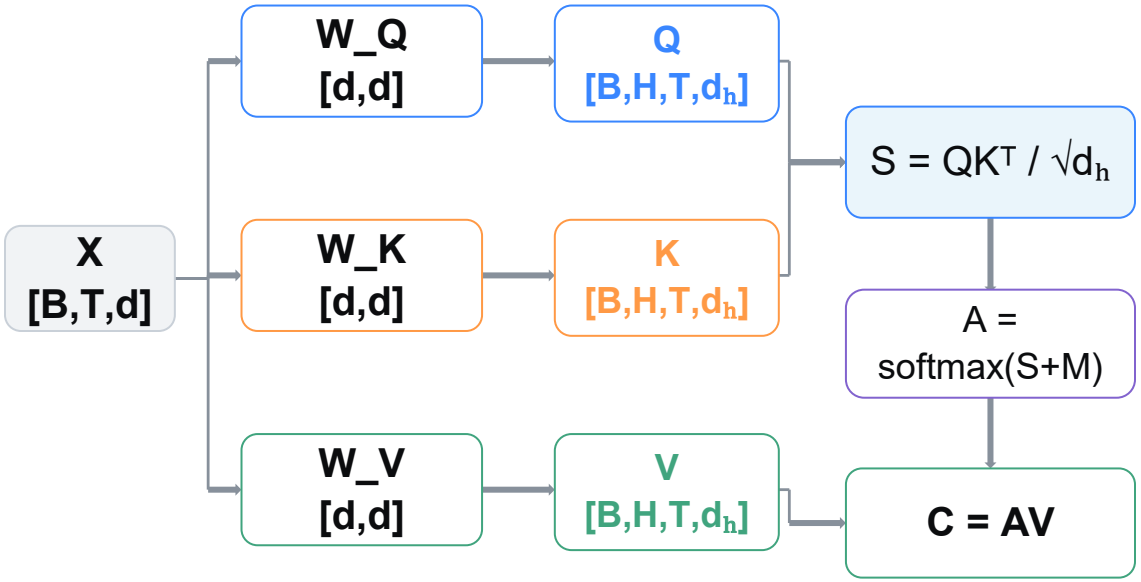
Quantization	Speed (tokens/s)	Speedup	Weight payload (GB)	Mean score (%)	Loss (pp)
BF16	80.45	1.00×	15.23	66.9	0.0
GPTQ INT8	114.32	1.42×	8.87	66.2	0.7
GPTQ INT4	143.40	1.78×	5.58	64.1	2.8
AWQ INT4	96.65	1.20×	5.57	64.1	2.8

Payload = bytes of stored tensors, including scales and unquantized tensors.
Mean score = reported mean of MMLU, C-Eval and IFEval. Loss is in percentage points.

Historical results. Payload excludes runtime memory. Quality uses separate task evaluations.

Qwen2 speed benchmark (2024) / Qwen2 quantized-model evaluation (2024) / Qwen official checkpoint tensor indexes

Recap of multi-head attention



Q and K include RoPE; M excludes future positions.

$$S = QK^T / \sqrt{d_h}$$

$$Q\ [1,4,16,32] \times K^T\ [1,4,32,16] = S\ [1,4,16,16]$$

For T=16, each head contains 16×16 pair scores.

Each row compares one query with all keys. Add M before softmax to hide future positions.

INPUT	OUTPUT	PARAMETERS	FLOPs
Q,K head tensors	S [1,4,16,16]	0	$2BT^2d$

Attention over a causal sequence

Example sequence The sky is blue

Keys, by token position

	The	sky	is	blue
q1	•	×	×	×
q2	•	•	×	×
q3	•	•	•	×
q4	•	•	•	•

$M[i,j] = 0$ when $j \le i$
$M[i,j] = -\infty$ when $j > i$

Each query combines only its prefix.
All prompt positions are already known, so their rows can run together.

The final row provides the last token's hidden state. A vocabulary projection predicts the next token.

Vaswani et al., Attention Is All You Need

The prompt is prefill, then decoding repeats

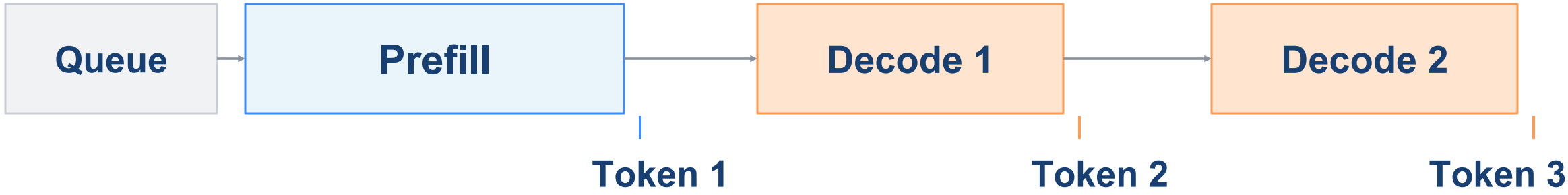
Illustrative tokens The sky is blue today .

Phase	New input to the model	Next token selected
Prefill	The sky is Positions 1–3 together	blue (position 4)
Decode step 1	blue Position 4 only	today (position 5)
Decode step 2	today Position 5 only	. (position 6)

Prefill produces the first generated token. Each decoding step consumes the last selected token to predict another.

NVIDIA, LLM Inference Optimization

What the user waits for



Time to first token (TTFT)

Queue + prompt processing
+ first-token selection

Time per output token (TPOT)

Repeated decode work
+ scheduling and delivery

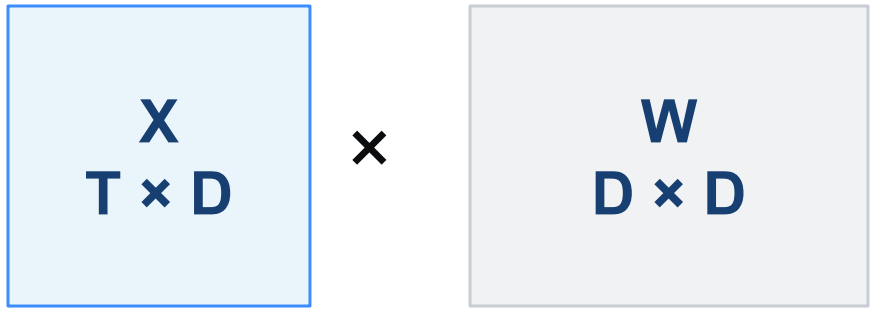
Batching can improve weight reuse and throughput. Every active request also adds KV reads and cache capacity.

JAX Scaling Book, Transformer Inference

The same weights, different amounts of reuse

One D × D projection, one request

Prefill



2TD² FLOPs
D² × bytes/weight to read W

Incremental decoding



2D² FLOPs
D² × bytes/weight to read W

Prefill reuses weights across P rows. Small-batch decoding uses one new row and reuses cached K/V for the prefix.

JAX Scaling Book, Transformer Inference

Why earlier keys and values remain valid

In causal inference, a token's hidden state depends only on its prefix.

At position t	Earlier positions 1 … t−1	New position t
Queries	Earlier Q is no longer needed	Compute q_t
Keys and values	Reuse cached K and V	Compute k_t and v_t
Attention output	Earlier rows stay unchanged	Compute the new output row

$$o_t = \text{softmax}(q_t\,[K_past\ ;\ k_t]^T / \sqrt{d})\,[V_past\ ;\ v_t]$$

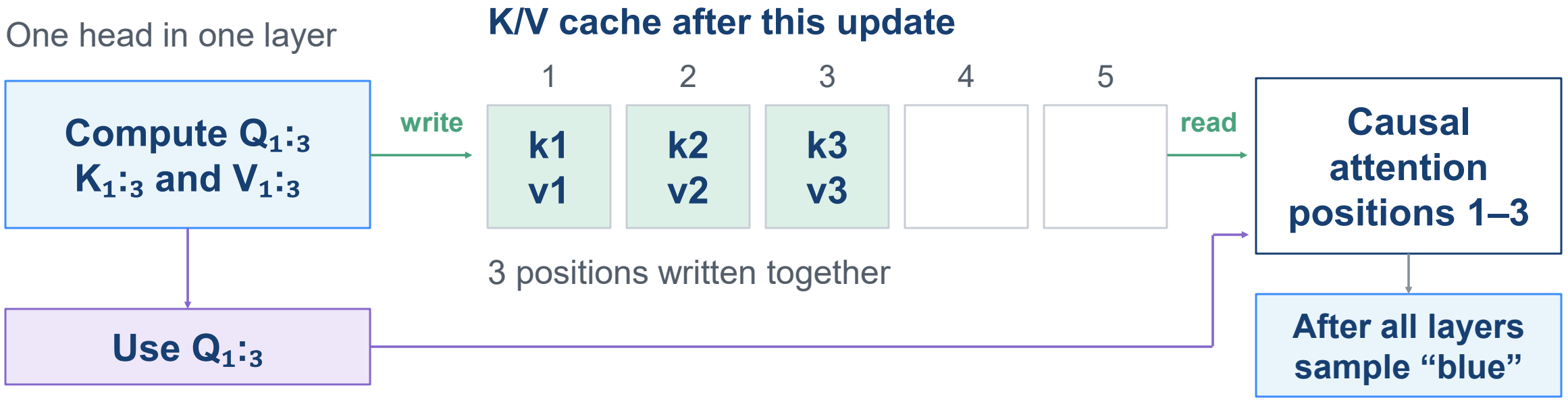
Fixed weights, fixed positions and an unchanged prefix make earlier K/V reusable at every layer.

Hugging Face, How caching works

Prefill fills the KV cache

Prefill processes positions 1–3

Input this step The sky is



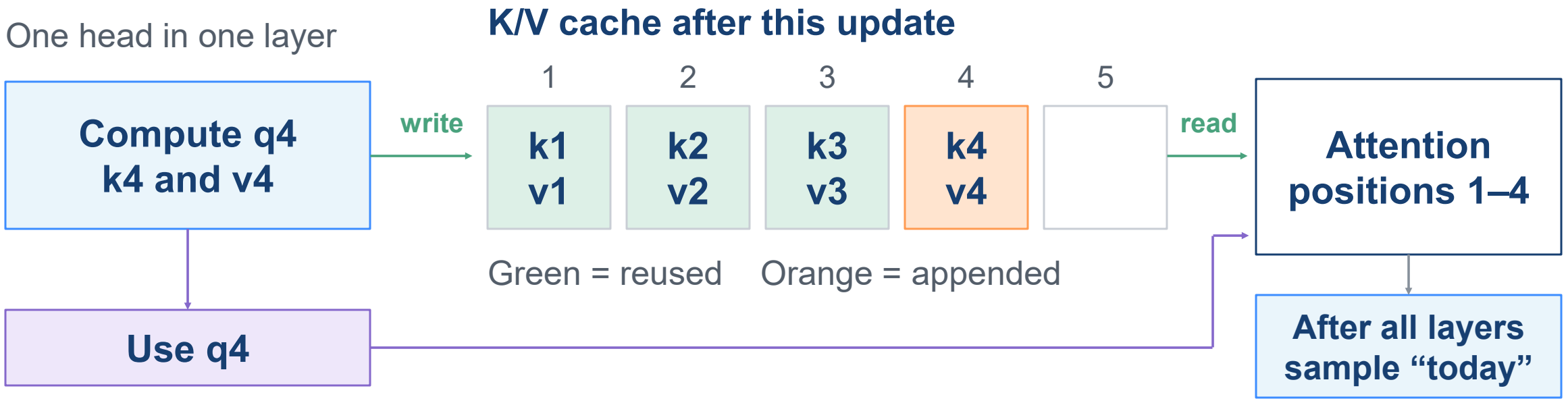
Prefill stores K and V for the prompt. Its last-position output produces the first generated token.

Decoding reads the cache and appends one token

Decode processes position 4

Input this step blue

One head in one layer



q4 attends to $K_{1:4}$ and $V_{1:4}$. Earlier Q values and earlier token computations are unnecessary.

KV belongs to a token and a layer

Each cell contains one token's K and V for all H heads

	Token 1	Token 2	Token 3	New token 4
Layer 1	K(1,1) V(1,1)	K(1,2) V(1,2)	K(1,3) V(1,3)	K(1,4) V(1,4)
Layer 2	K(2,1) V(2,1)	K(2,2) V(2,2)	K(2,3) V(2,3)	K(2,4) V(2,4)
Layer L	K(L,1) V(L,1)	K(L,2) V(L,2)	K(L,3) V(L,3)	K(L,4) V(L,4)

Each layer has its own projection weights and hidden states, so its cache entries differ.

Hugging Face, How caching works

How large does the KV cache become?

MHA stores K and V per layer as [B, H, T, d]. B is the number of requests.

$$\text{KV bytes} = 2 \times L \times B \times T \times H \times d \times \text{precision}$$

32 layers, 32 heads, d = 128, BF16
Per token per request $2 \times 32 \times 32 \times 128 \times 2 = 512$ KiB

Context length	One request	Eight requests
4,096 tokens	2 GiB	16 GiB
8,192 tokens	4 GiB	32 GiB

Full attention, equal lengths, no allocation overhead. Weight quantization leaves KV bytes unchanged unless KV precision also changes. GQA, MQA and MLA reduce the attention state in different ways.

Hugging Face, How caching works / Shazeer, Fast Transformer Decoding

FLOPs in prefill and one decoding step

One layer, one request. Decoding uses cached earlier K and V.

Operation	Prefill, P input tokens	Decode, T context tokens
Q, K and V projections	$6PD^2$	$6D^2$
QK^T and attention × V	$2P(P + 1)D$	$4TD$
Output projection WO	$2PD^2$	$2D^2$
SwiGLU FFN, width F	$6PDF$	$6DF$

A multiply-add counts as 2 FLOPs. Multiply by the number of layers L.
T includes the new token. Attention counts useful causal pairs.

Counts exclude softmax, normalization and elementwise work. Full-square attention adds masked work.

Shazeer, Fast Transformer Decoding

Data I/O in prefill and one decoding step

One layer, one request. I/O here means traffic to and from GPU HBM.

Data movement	Prefill	Decode
Read projection + FFN weights	$N\ell \times bW$	$N\ell \times bW$
Read earlier cached K/V	No earlier cache	$2(T-1)D \times bKV$
Write new K/V to the cache	$2PD \times bKV$	$2D \times bKV$

$N\ell = 4D^2 + 3DF$ weights per layer. bW and bKV are bytes per value.
This is a minimum of one weight read and one read/write per cache element.

Activations and attention intermediates add traffic. Materializing P × P attention matrices adds quadratic I/O.

Shazeer, Fast Transformer Decoding / Dao et al., FlashAttention

A worked cost estimate

32 layers, MHA, D = 4,096, H = 32, F = 11,008
P = T = 4,096, batch 1, BF16 weights and KV (2 bytes each)

Across the Transformer blocks	Prefill	One decode step
Projection + FFN parameters	6.48 billion	6.48 billion
FLOPs, including causal attention	57.45 TFLOPs	15.10 GFLOPs
Weight reads	12.95 GB	12.95 GB
New KV writes / old KV reads + writes	2.15 GB	2.15 GB
Weight + KV traffic floor	15.10 GB	15.10 GB

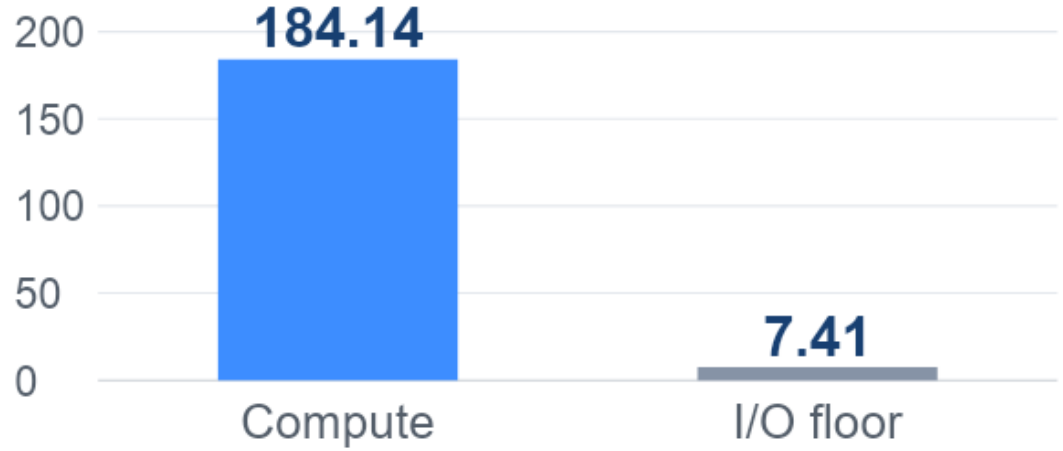
Calculated model, not a benchmark. Decimal GB. Embeddings, vocabulary head, other activations, intermediates and runtime overhead are excluded. Equal P and T compare two separate stage inputs.

Compute bound and memory bandwidth bound

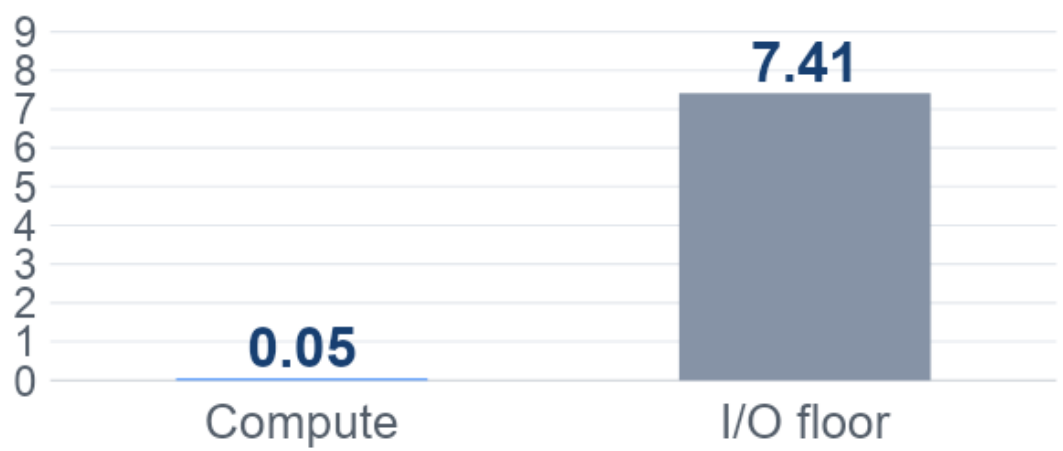
A100 80GB SXM, dense BF16 peak 312 TFLOP/s, HBM 2,039 GB/s

Time ≥ max(FLOPs / compute rate, bytes / bandwidth)

4,096-token prefill (ms)



One decode step (ms)

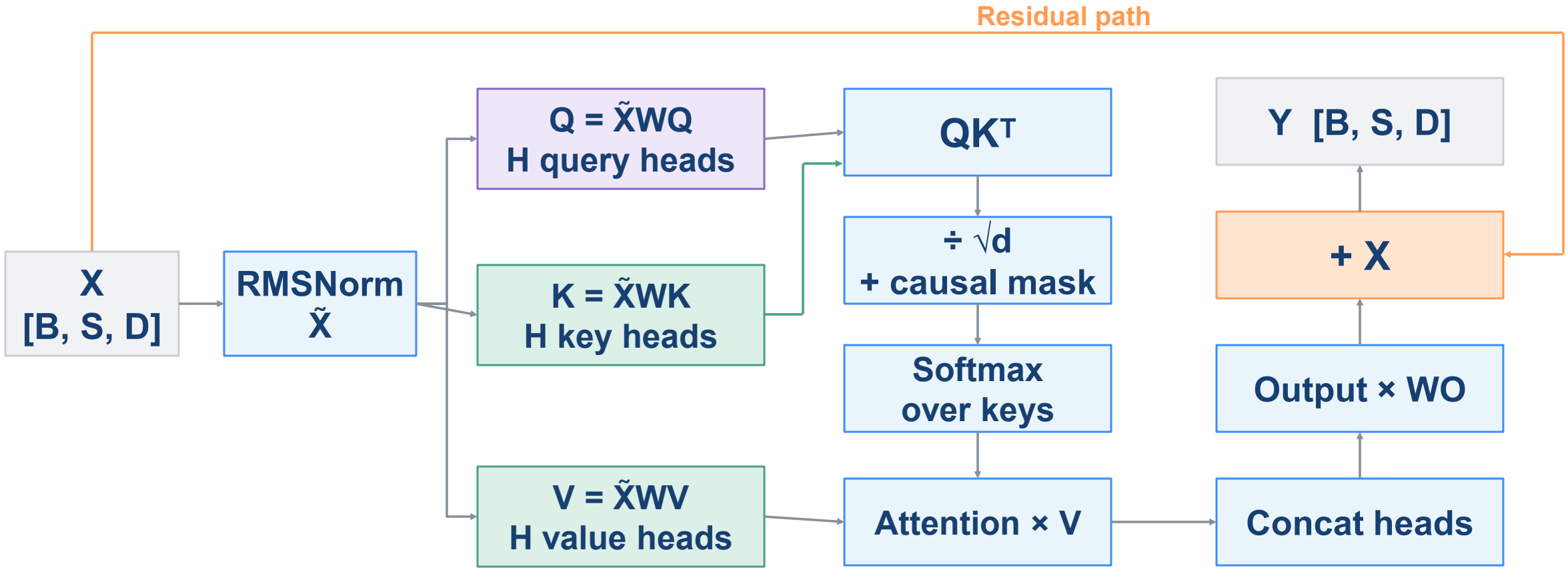


Calculated floors from the previous slide. The plots use different scales. Actual I/O and latency are higher.

Long prefill is often compute bound. Small-batch decoding is often bandwidth bound. Short prompts, batch size and kernels can change the bottleneck.

NVIDIA A100 datasheet, 80GB SXM / JAX Scaling Book, Transformer Inference

Multi-head attention (MHA)



B requests, S input tokens, H heads, d = D/H. Each head runs its own attention.

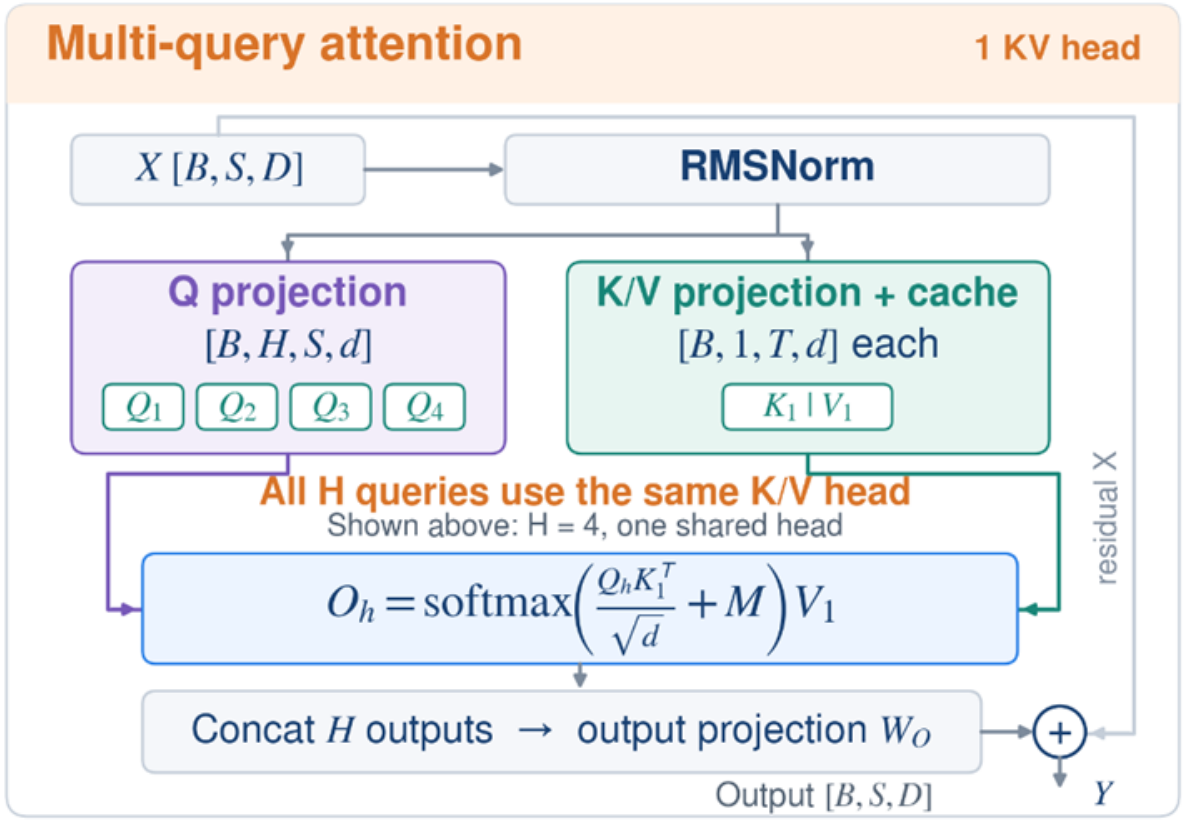
Attention sublayer only. RoPE omitted. The FFN follows this residual in a complete Transformer block.

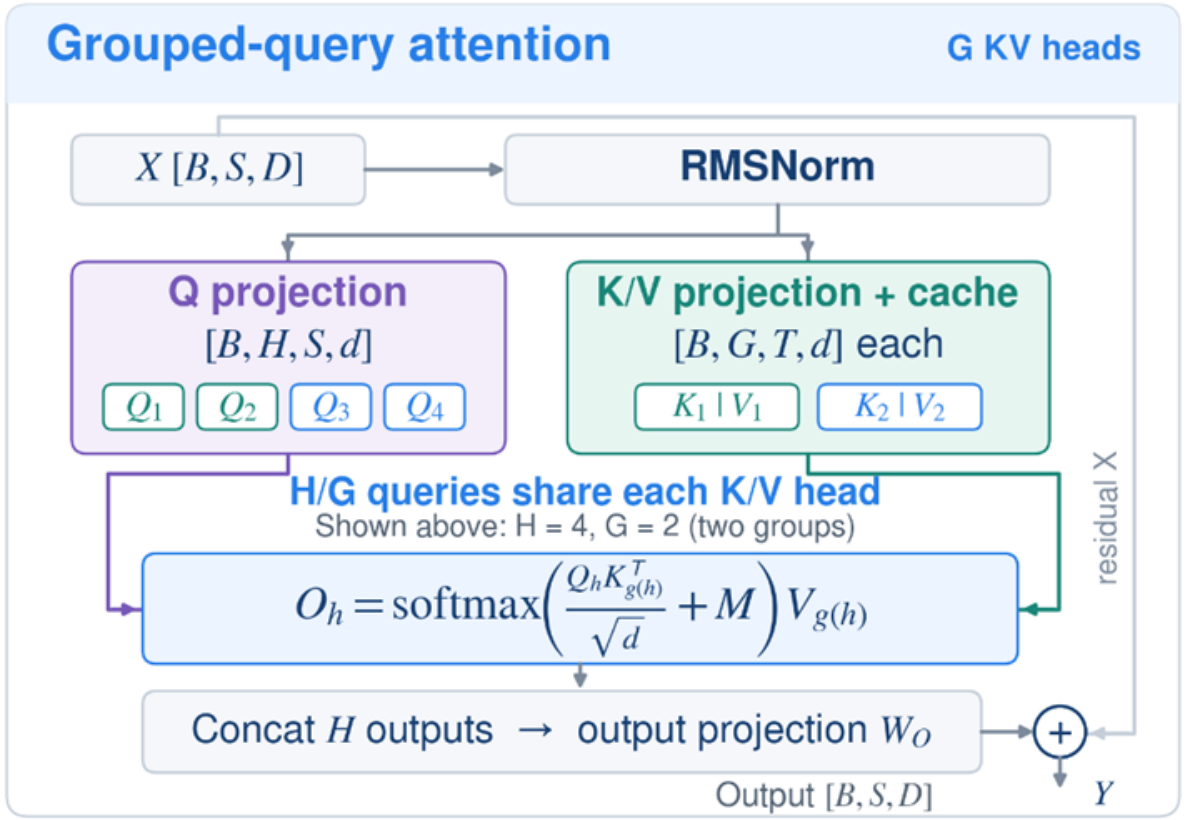
Vaswani et al., Attention Is All You Need / Meta, The Llama 3 Herd of Models, Table 3

MQA and GQA

Separate query heads. Shared K/V heads.

MQA shares one K/V head across all queries; GQA shares one K/V head within each group.





B batch · S query tokens · T cached tokens · H query heads · G KV groups · d head size · D = Hd · M causal mask · g(h) group of head h

Shazeer, Fast Transformer Decoding / Ainslie et al., GQA H query heads, G K/V heads, d = D/H. S = 1 in decoding. T includes cached tokens.

Cache savings exceed FLOP savings

Same dimensions as the exercise. Only the K/V head count changes.

Measure	MHA, G=64	GQA, G=8	MQA, G=1
Projection weights / layer	512 MiB	288 MiB	260 MiB
Projection weight saving	Baseline	43.75%	49.22%
Prefill FLOPs / layer	2.474 TF	1.512 TF	1.392 TF
Decode FLOPs / layer	671.09 MF	436.21 MF	406.85 MF
KV cache / 80 layers	10 GiB	1.25 GiB	160 MiB

GQA-8 **8× smaller KV cache, 35% fewer decode attention FLOPs**

QK^T and attention × V still run for all 64 query heads.

Calculated matmul FLOPs include Q/K/V/O projections and attention. Excludes FFN, norms, RoPE and softmax.

Memory: closer is faster, but smaller

CPU teaching example: registers → caches → DRAM → storage

Where data lives	Illustrative access cost	Role
Registers	Already in the core	Operands for instructions
L1 cache (SRAM)	~4 CPU cycles	Very small, very fast
L2 cache (SRAM)	~10 CPU cycles	Larger on-chip cache
Main memory (DRAM)	~100 CPU cycles	Working data and model weights
SSD storage	50 µs ≈ 150,000 cycles*	Persistent files
Hard disk storage	~10,000,000 cycles	Persistent files

*SSD conversion assumes a 3 GHz CPU. These are teaching estimates, not a specification for a modern CPU or GPU.

CMU 15-213, Memory Hierarchy (2015)

One addition can spend most time waiting

Toy dependency: load one operand, then perform a 1-cycle integer add.

Operand from L1	Load: 4 cycles	Add: 1	5 cycles
Operand from DRAM	Load: 100 cycles	Add: 1	101 cycles

The dependent add must wait until its operand arrives.

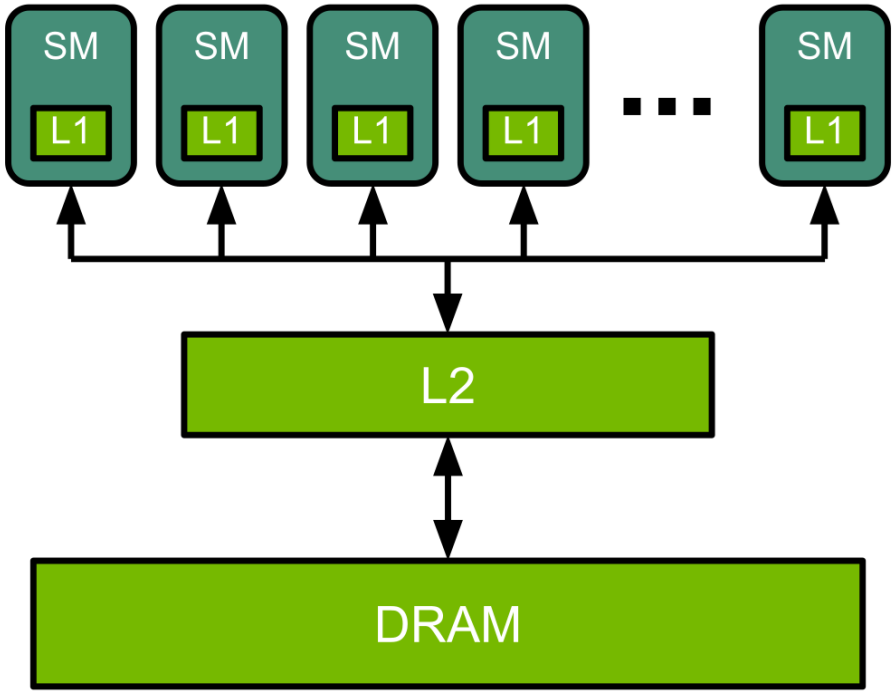
- Reuse data near the compute units to avoid fetching it repeatedly.
- Overlap independent work to hide latency; bandwidth still limits bytes/s.

Illustrative serial path, not to scale. Pipelines, prefetching and parallel work are omitted.

CMU 15-213, Memory Hierarchy (2015) / NVIDIA, GPU Performance Background

A GPU moves tiles to many compute units

NVIDIA A100 example: 108 streaming multiprocessors (SMs)



- Read tiles from HBM through L2 into fast on-chip storage.
- SMs compute: tensor cores handle matrix products; other units do reductions.
- Warps contain 32 threads. Ready warps run while others wait for data.

HBM = high-bandwidth GPU DRAM

Reuse a tile in shared SRAM / registers before loading the next tile.

NVIDIA, GPU Performance Background / NVIDIA, Ampere Architecture

Latency versus throughput

Latency: time for one request. Throughput: completed requests per second.



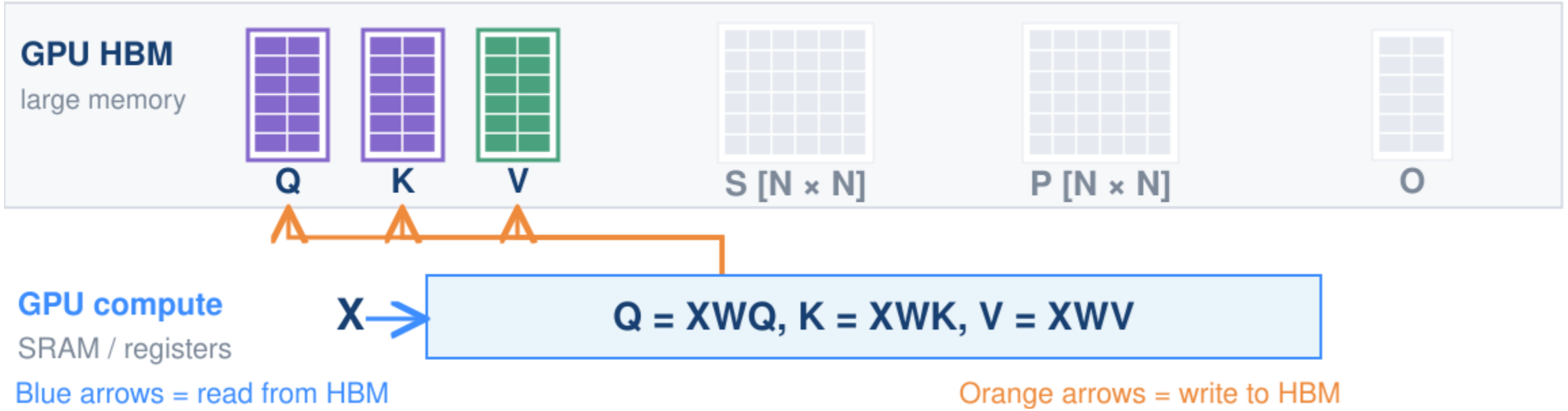
Fully busy for 1 second	Request latency	Completed in 1 second
One worker	10 ms	1,000 / 10 = 100
Four concurrent workers	20 ms	4 × (1,000 / 20) = 200

Assume four workers can run concurrently, no queueing and negligible overhead.

Bottleneck 1: MHA score matrices

32-head MHA, N = 8,192 tokens, BF16: S alone occupies 4 GiB.

1 / 7 Project the input X into Q, K and V for this head.



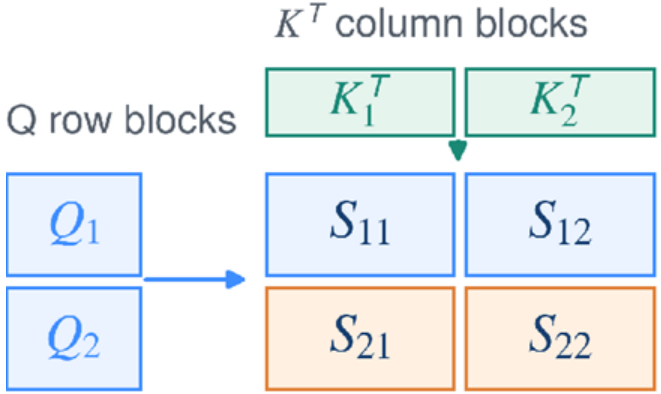
$$S_h = \frac{Q_h K_h^T}{\sqrt{d}} + \text{mask}, \quad P_h = \text{softmax}_{\text{rows}}(S_h), \quad O_h = P_h V_h$$

Dao et al., FlashAttention (2022)

Tile-wise computation and softmax

A softmax row shares one maximum and one denominator across all its key blocks.

1 QK^T in parallel tiles



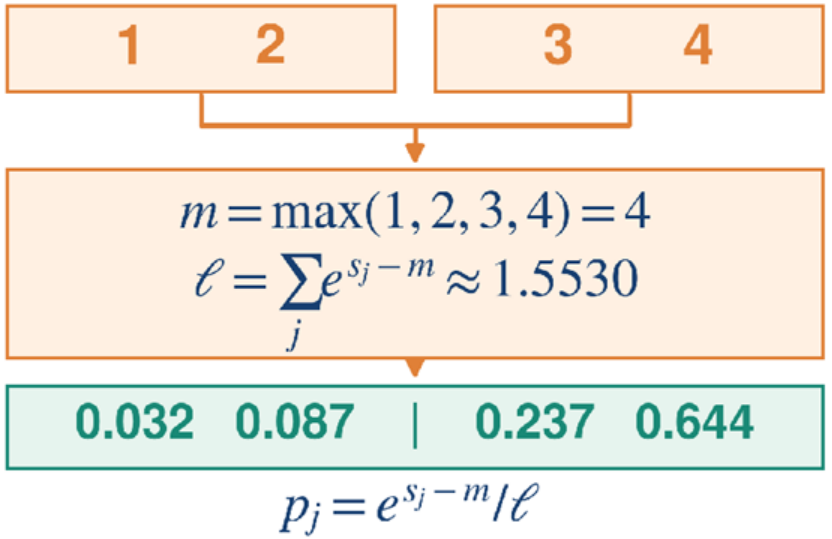
$$S_{ab} = Q_a K_b^T / \sqrt{d}$$

Split Q rows and K^T columns.

Each tile reduces over features d.

2 Softmax across one row

One query, four allowed key positions



$$p_j = e^{s_j - m} / \ell$$

3 Multiply by V

$$O = PV$$



Tile output rows and columns.
Run those tiles in parallel.

Each tile reduces over keys.

Different query rows and attention heads remain independent.

Head 1 $QK^T \rightarrow \text{softmax} \rightarrow PV$

Head 2 $QK^T \rightarrow \text{softmax} \rightarrow PV$

Head H $QK^T \rightarrow \text{softmax} \rightarrow PV$

Row reductions can run in parallel. Online softmax merges statistics across tiles.

Original and online softmax compute the same attention output

One query: s_i is an attention score, v_i is a value vector, and B is the next block of tokens.

Original · normalize the whole row

All N scores contribute to the denominator.

$$m = \max(s_1, \ldots, s_N)$$

$$Z = \sum_{j=1}^{N} e^{s_j - m}$$

$$p_i = \frac{e^{s_i - m}}{Z}, \qquad o = \sum_{i=1}^{N} p_i v_i$$

Separate kernels store scores / probabilities in HBM, then read them again to combine V.

Online · update one block at a time

m = max Z = weight sum U = weighted value sum

$$m' = \max\left(m, \max_{i \in B} s_i\right) \qquad \alpha = e^{m - m'}$$

$$Z' = \alpha Z + \sum_{i \in B} e^{s_i - m'}$$

$$U' = \alpha U + \sum_{i \in B} e^{s_i - m'} v_i$$

$$o = U/Z$$ after the final block

Rescaled old summaries + **next block's contribution**

Initialize from the first block; replace m, Z, U after each update.

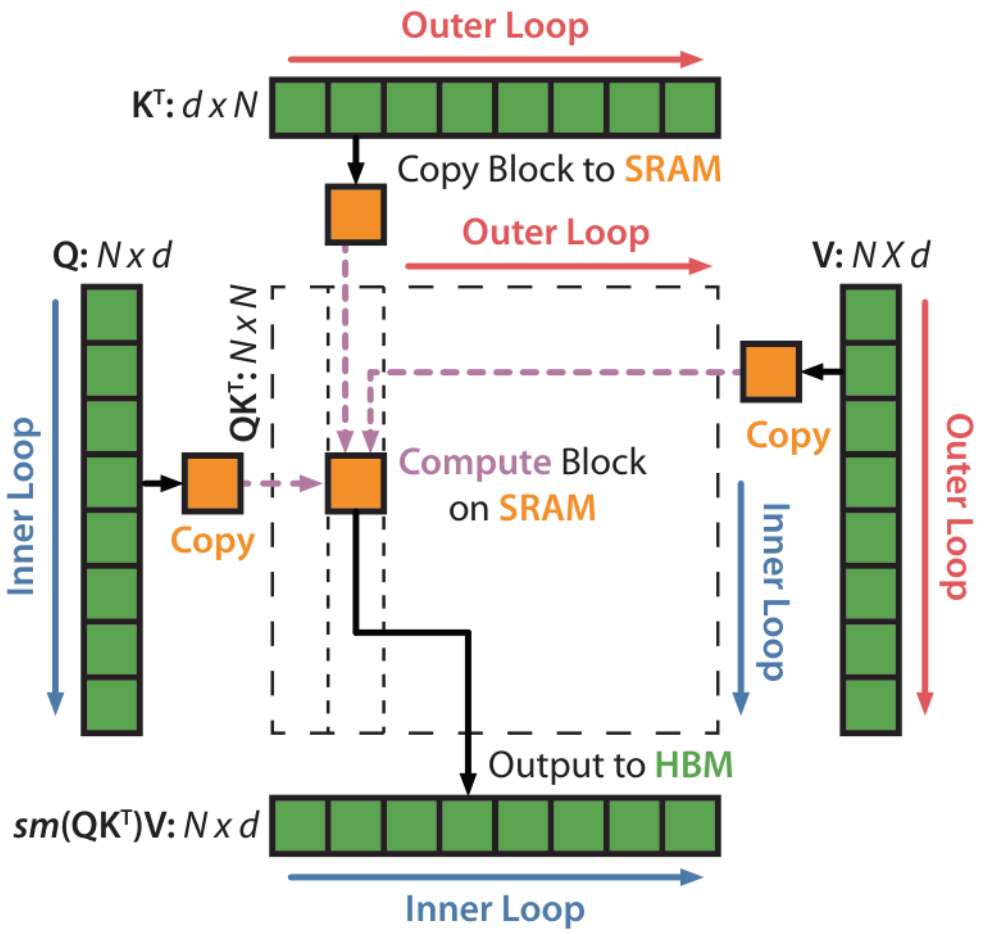
Why rescale? When the maximum changes, α adjusts the old sums without rereading old scores.

Why less memory? Fused attention keeps (m, Z, U) + one block in SRAM, then discards the block's scores.

No full N × N score / probability matrices are written to HBM.

Online softmax + value aggregation, as used in FlashAttention. Same algebra; floating-point rounding may differ. Source: Dao et al. (2022), §3.1.

FlashAttention



Online softmax combines the results of small blocks.

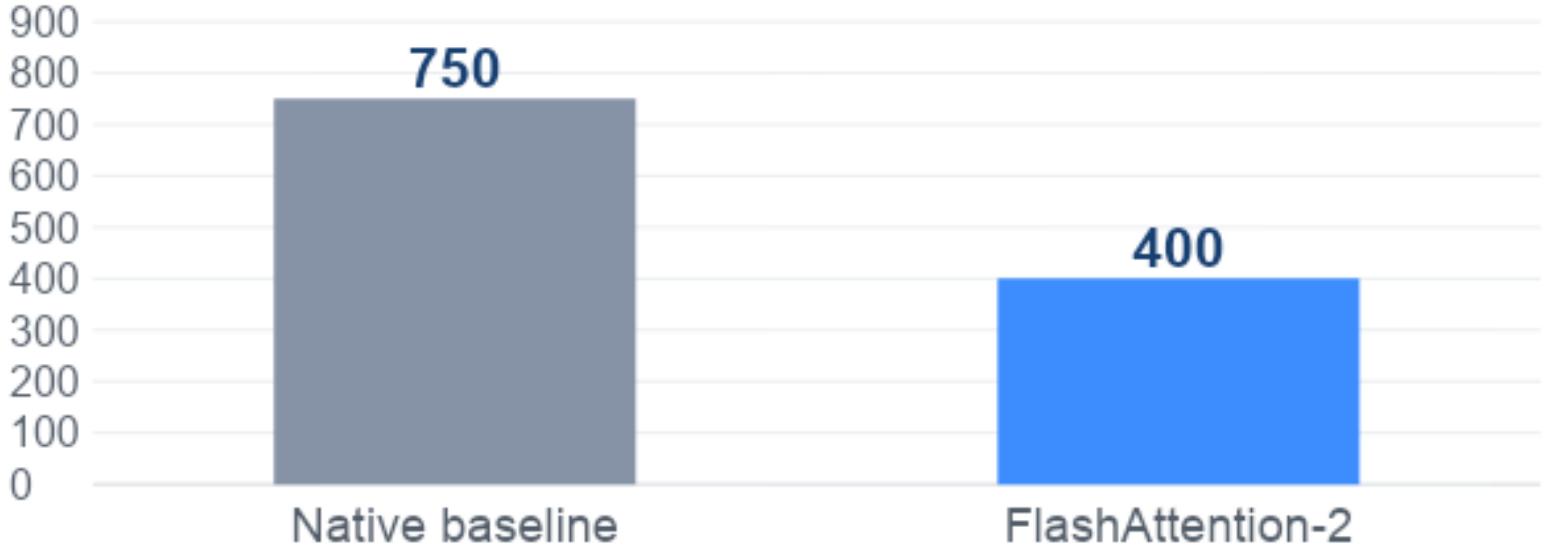
1. Load one K/V block into SRAM (red outer loop).
2. Reuse it for blocks of Q (blue inner loop).
3. Compute scores and softmax (sm) on chip.
4. Rescale and update the output using online softmax.

Dao et al., FlashAttention (2022)

FlashAttention-2 reduces forward latency

Llama-2-7B • NVIDIA A100 • batch 1 • 4,096-token unpadded input

One forward pass (ms), lower is better



≈1.9×

faster forward pass

≈47% lower latency

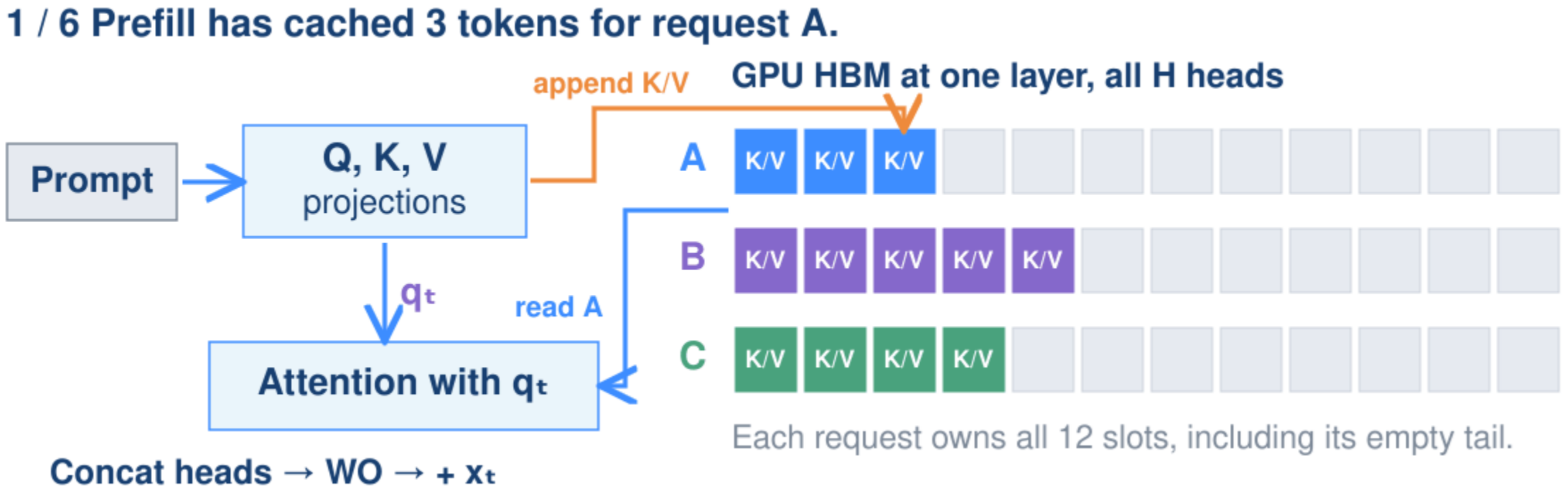
Rounded readings from the published chart: about 750 ms → 400 ms.

This measures a long-prompt forward pass; generation and request queueing are excluded.

Hugging Face, Transformers v4.39.3 benchmark / Dao, FlashAttention-2 (2023)

Bottleneck 2: the growing MHA KV cache

Each new token adds K/V. Its query reads the request's full cached history.



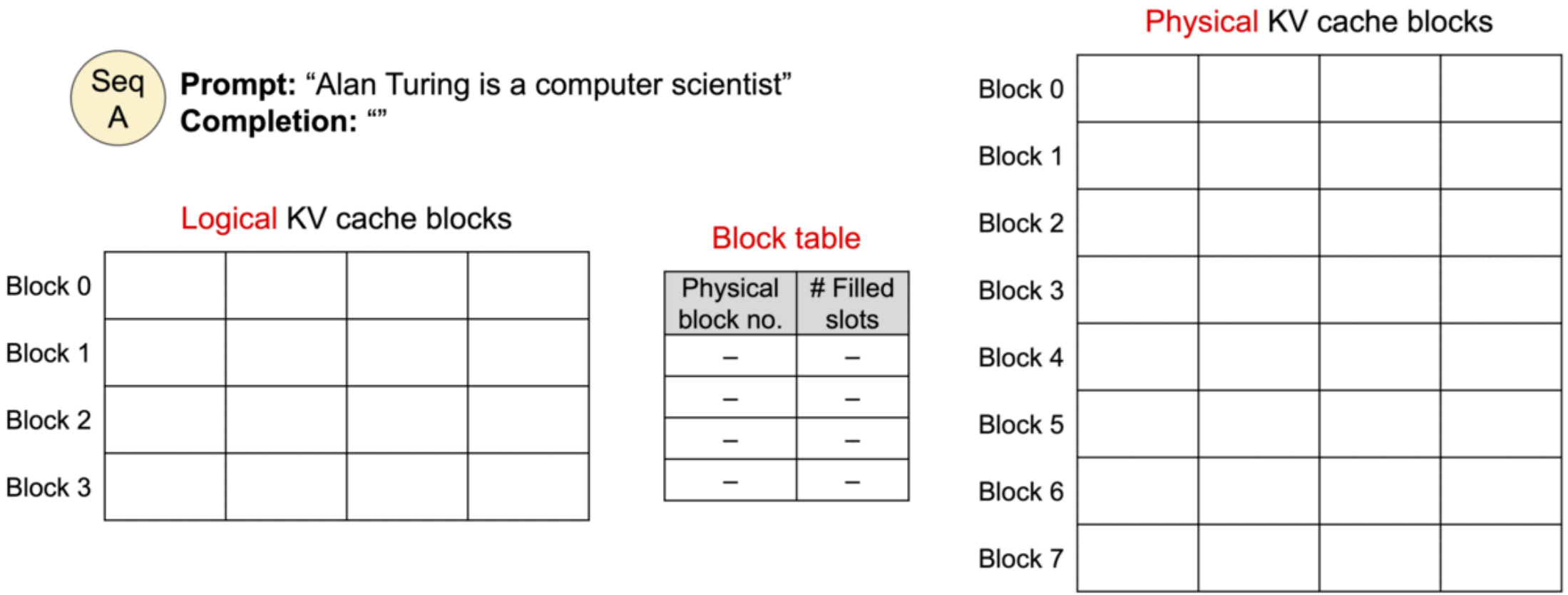
$$\text{Useful KV bytes} = 2LHdb\sum_i T_i$$

L layers, H heads, d values/head
b bytes/value, T_i tokens in request i

Kwon et al., PagedAttention (2023) / vLLM, PagedAttention launch results (2023)

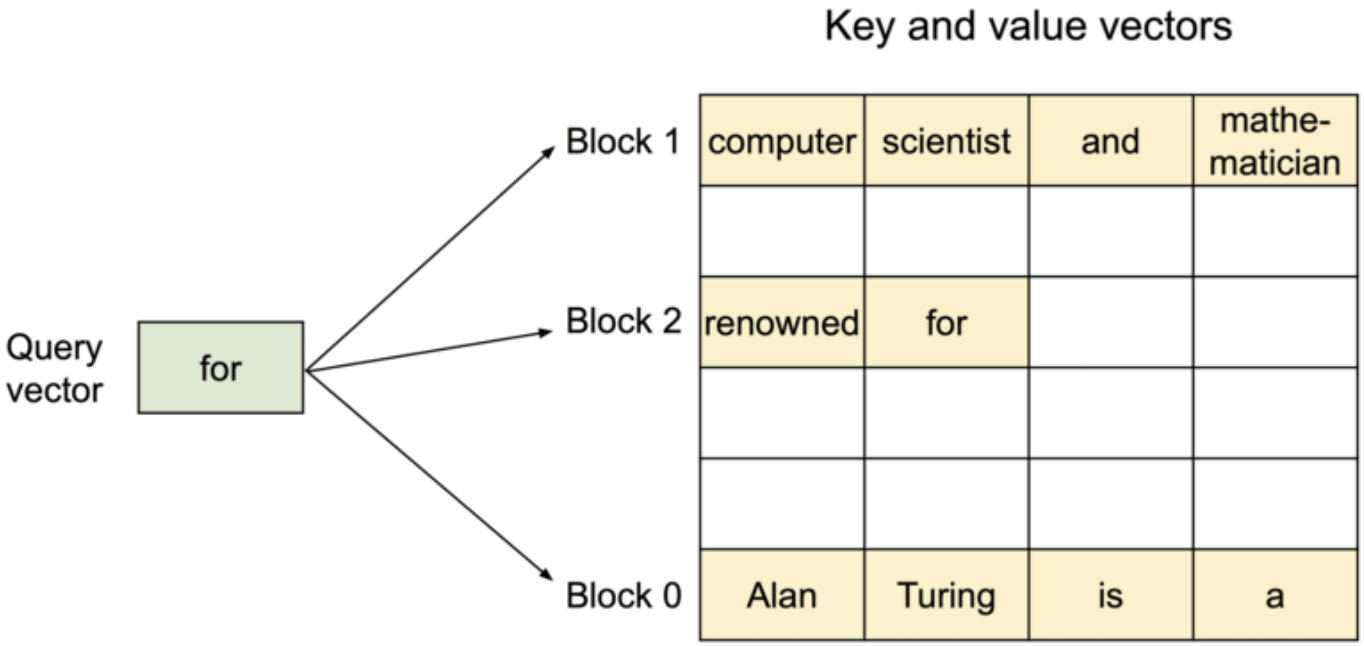
PagedAttention allocates KV blocks as needed

Follow the growing sequence, its block table and the physical GPU blocks.



vLLM, PagedAttention launch results (2023) / Kwon et al., PagedAttention (2023)

PagedAttention reads scattered KV blocks



1 Locate blocks

Follow this request's block table.

2 Read K/V

Normalize scores across all blocks and combine V.

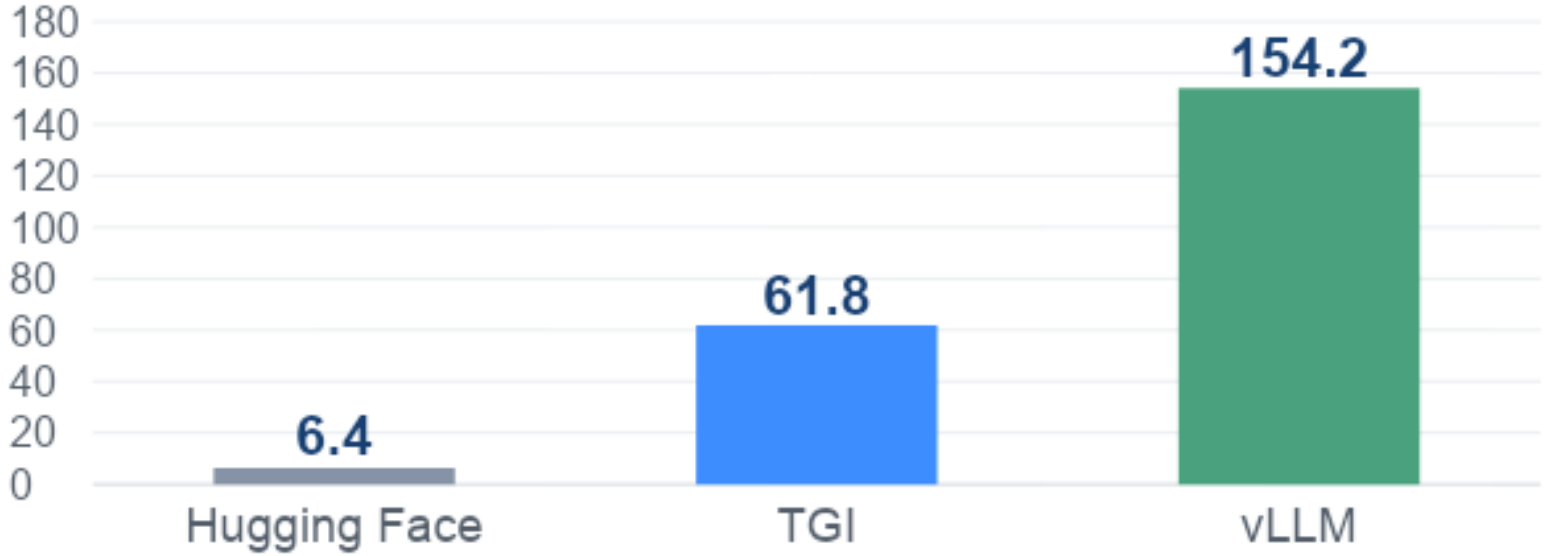
Block labels 0, 1 and 2 denote logical order; their physical locations can differ.

vLLM, PagedAttention launch results (2023) / Kwon et al., PagedAttention (2023)

Paged KV storage enables higher throughput

LLaMA-13B • A100 40 GB • ShareGPT request lengths • one completion

Serving throughput (requests/min), higher is better



2.5×

throughput vs TGI

154.2 / 61.8

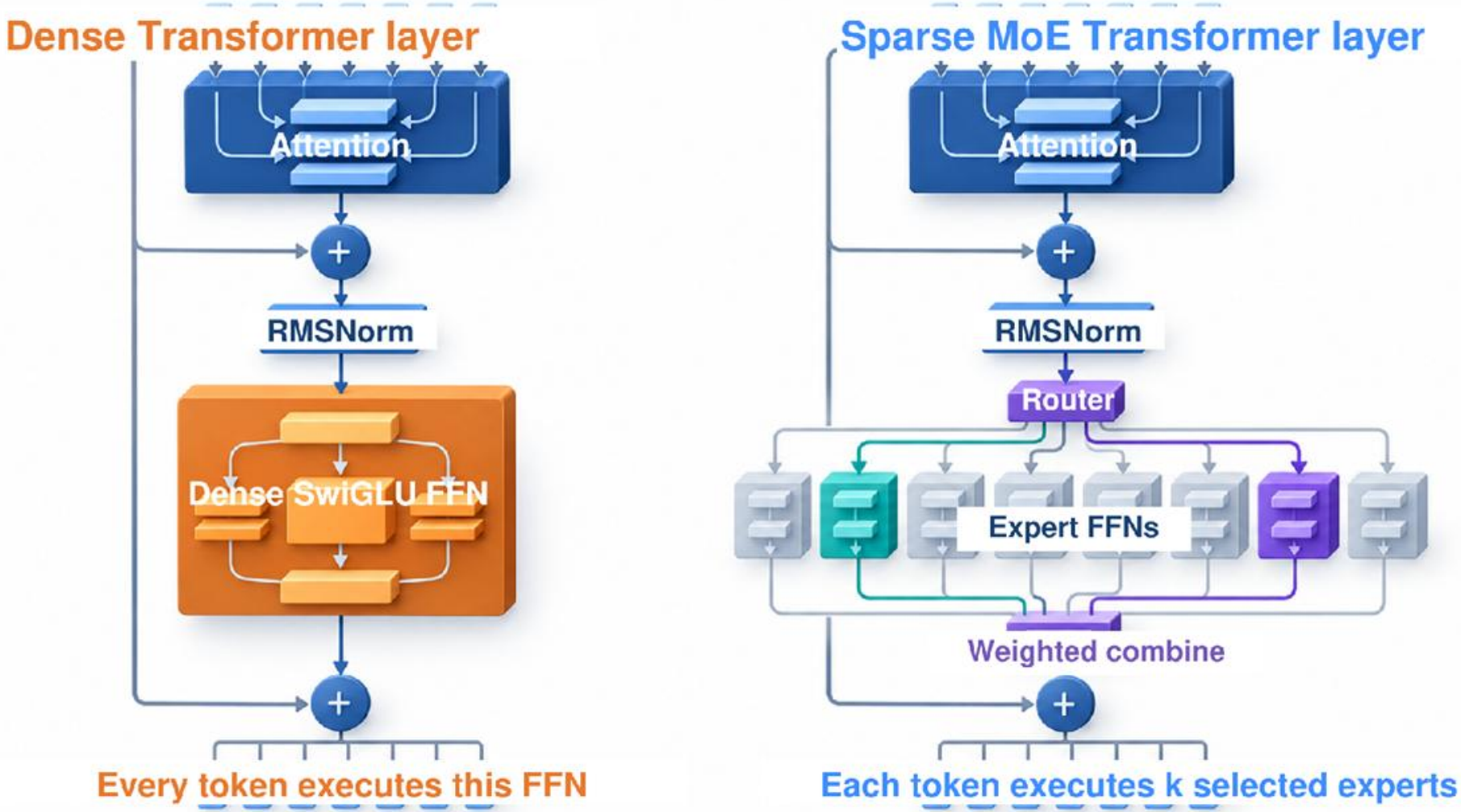
Less KV waste → more requests fit → larger effective batches.

Published 2023 engine comparison. Includes scheduling and kernels, beyond PagedAttention alone.

vLLM, PagedAttention launch results (2023) / Kwon et al., PagedAttention (2023)

Dense FFN and sparse MoE

MoE replaces the feed-forward sublayer inside a Transformer block. The attention path stays unchanged.



Dense and sparse blocks both keep residual connections around attention and the FFN/MoE sublayer.

Sources: Shazeer et al. (2017); Jiang et al. (2024), Mixtral of Experts.

Top-k routing in an MoE layer

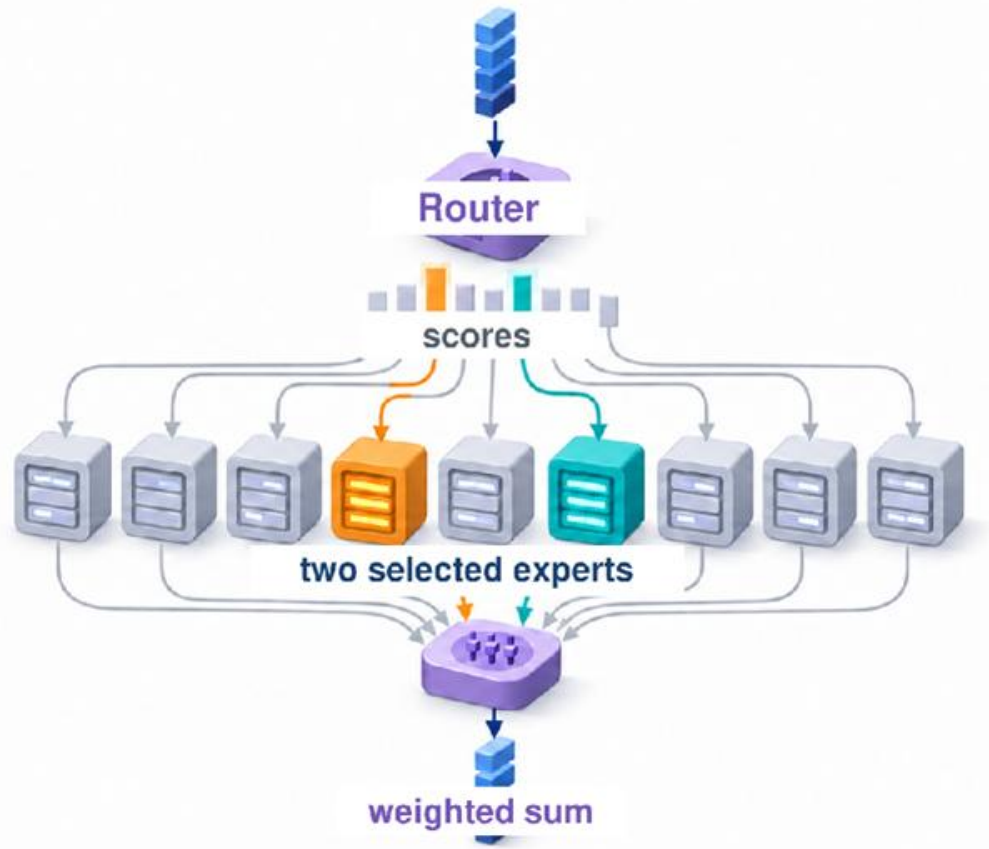
For each token state x, the router produces one logit for every expert. Top-k keeps only the selected routes.

$$\textbf{Router logits:} \quad \mathbf{z} = W_r \mathbf{x} + \mathbf{b}_r, \qquad W_r \in \mathbb{R}^{E \times D}$$

$$\textbf{Routing probabilities:} \quad p_i = \frac{e^{z_i}}{\sum_{j=1}^{E} e^{z_j}}, \qquad \sum_{i=1}^{E} p_i = 1$$

$$\textbf{Sparse gate:} \quad \mathcal{T}_k(\mathbf{x}) = \mathrm{TopK}(\mathbf{p}, k), \qquad g_i = \begin{cases} \dfrac{p_i}{\sum_{j \in \mathcal{T}_k} p_j}, & i \in \mathcal{T}_k \\ 0, & \text{otherwise} \end{cases}$$

$$\textbf{MoE output:} \quad \mathrm{MoE}(\mathbf{x}) = \sum_{i \in \mathcal{T}_k(\mathbf{x})} g_i \, E_i(\mathbf{x}).$$



The router is a learned linear layer that maps each token to E expert logits.

Worked top-2 example

z = [2, 1, 0, −1]

p = [0.644, 0.237, 0.087, 0.032]

Top-2 selects E_1 and E_2

Renormalized gate: g = [0.731, 0.269, 0, 0]

MoE(x) = 0.731 E_1(x) + 0.269 E_2(x)

During training, a load-balancing loss discourages too many tokens from choosing the same expert.

Sources: Shazeer et al. (2017); Jiang et al. (2024).

Total and active parameters

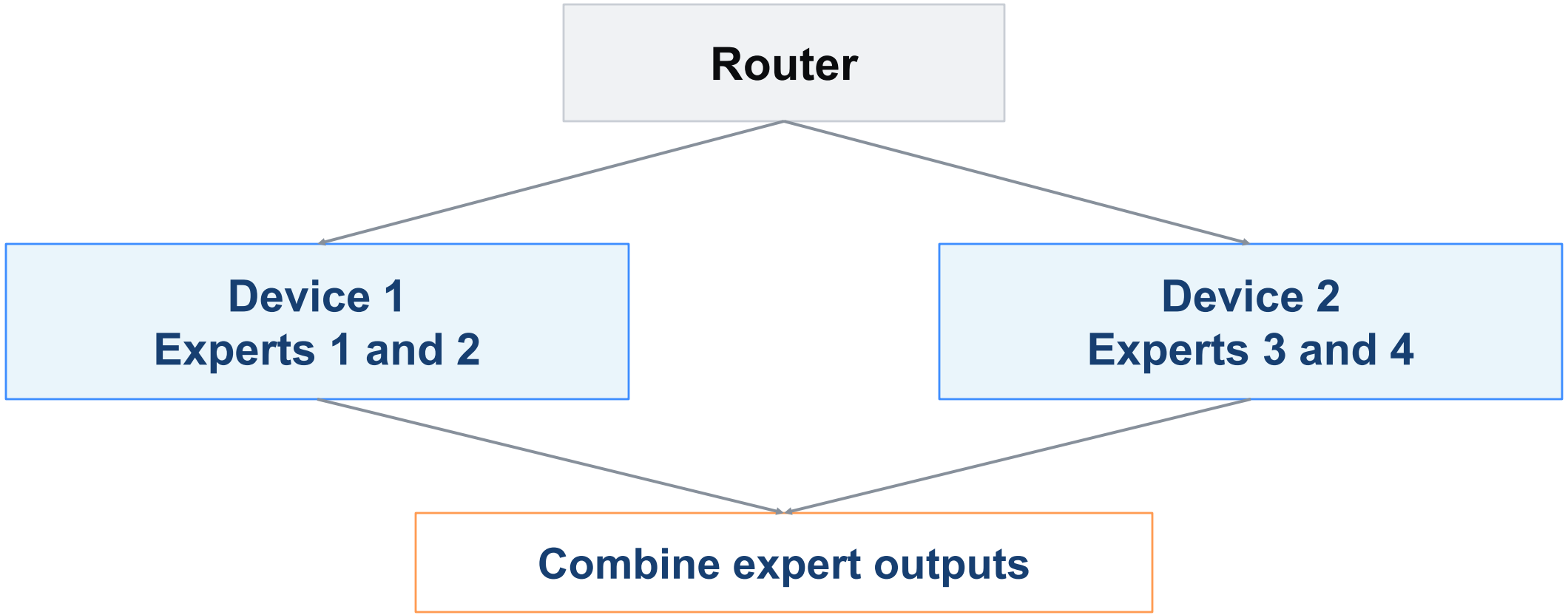
Published model	Total parameters	Active per token
Mixtral 8×7B	46.7B	12.9B
DeepSeek-V3	671B	37B

- Total parameters influence stored weight capacity.
- Active parameters help describe per-token work.

Mixtral 8×7B shares attention and other components, so its total is not simply 8 × 7B.

Jiang et al., Mixtral of Experts (2024) · DeepSeek-V3 report (2024/2025)

Expert parallelism moves token vectors



Communication and uneven expert load can limit speed even when active computation is small.

Serving an MoE model

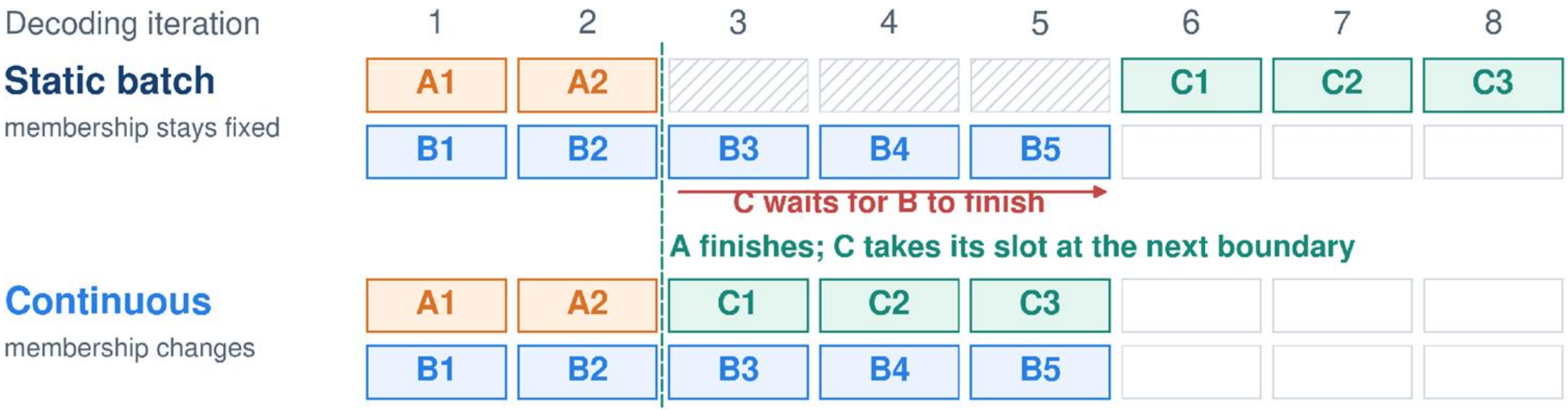
Deployment issue	What to inspect
Many distinct experts in one batch	Expert coverage and weight traffic
Popular experts receive more tokens	Load balance and overloaded devices
Experts spread across devices	Interconnect bandwidth and communication time
Experts offloaded to host memory	Transfer volume, reuse and scheduling

Continuous batching: refill freed slots

Reuse a free slot at the next iteration boundary; other requests continue.

Two decode slots. A needs 2 more steps, B needs 5, C needs 3.

C becomes decode-ready before step 3; its prompt prefill is already complete.



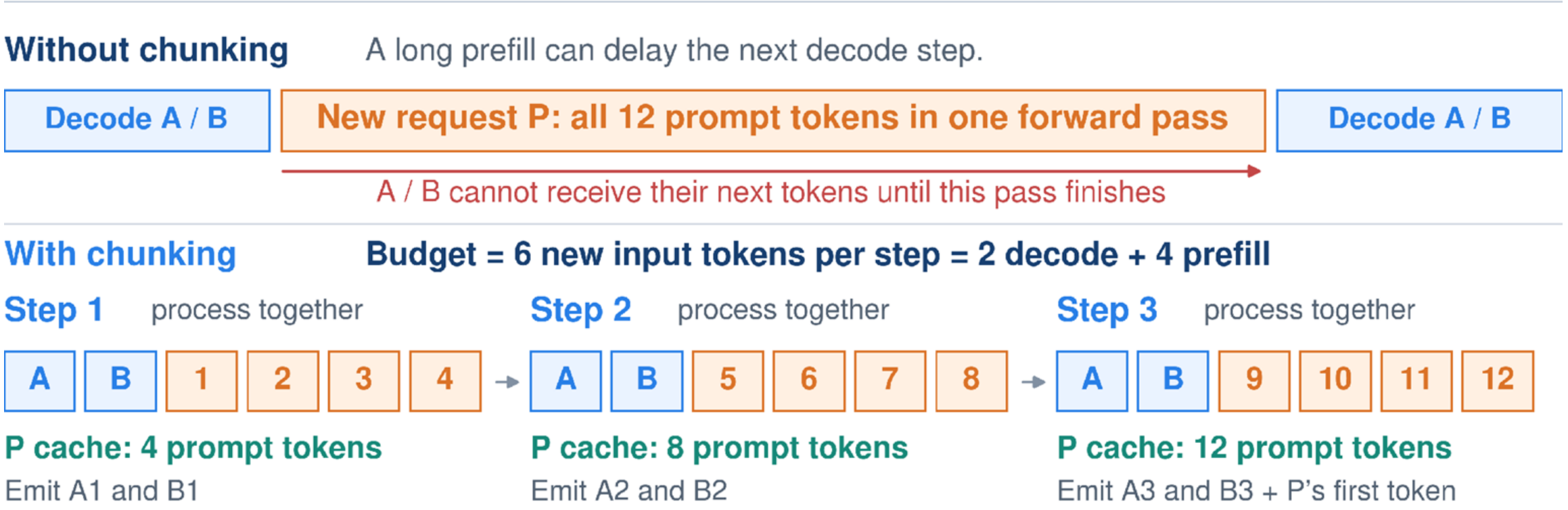
Same requests complete by step 5 instead of step 8 in this schematic.

- Schedule at token-step boundaries, not only after an entire batch finishes.
- Steps are schematic: step time may change with the batch, so this is not a measured speedup.

Orca, OSDI 2022. Illustrative request lengths; not a latency benchmark.

Chunked prefill: share each serving step

Split a long prompt into pieces so ongoing requests can keep receiving tokens.



P finishes prefill at step 3. Its first generated token can join decoding in step 4.

- Decode-first example: reserve 2 tokens for A/B, then use the remaining 4-token budget for P.
- Chunk size trades prompt completion time against decode interruptions; tokens are not a time budget.

SARATHI (2023); Sarathi-Serve, OSDI 2024; vLLM optimization documentation. Schematic schedule.

Prefix caching

Two requests share an exact token prefix

System prompt + shared document	Question A
System prompt + shared document	Question B

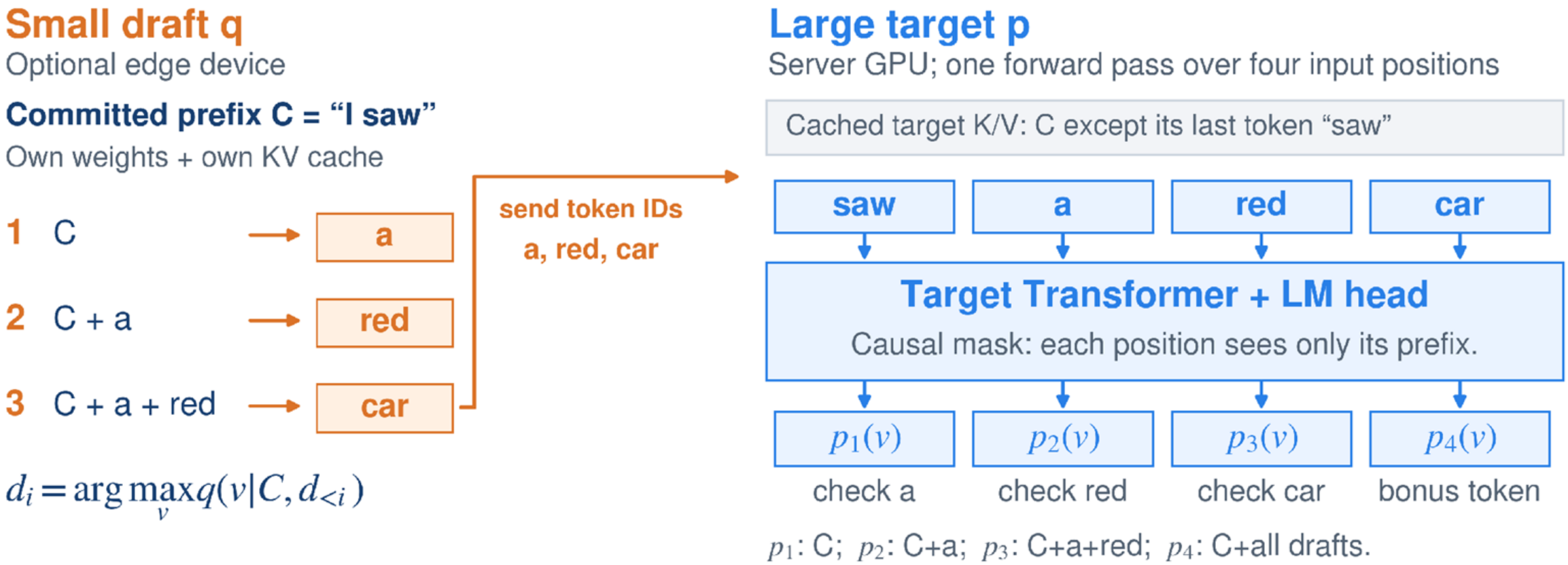
Reuse the shared prefix’s KV state when the cache contains a valid match.

Prefix caching saves repeated prefill work. Each new answer still requires decoding.

vLLM automatic prefix caching

Speculative decoding: draft, then verify

A small model proposes three tokens; the large model checks them in one causal forward pass.



- Drafting is autoregressive. The target checks all positions together without future-token leakage.
- Each model owns its K/V cache. An edge draft adds a network round trip per speculative round.

Leviathan et al., ICML 2023; Hugging Face, Assisted Generation (2023). Greedy, shared-tokenizer example.

Speculative decoding: accept or roll back

Greedy example: accept matching tokens in order and stop at the first mismatch.

Greedy rule: accept d_i only if $d_i = \arg\max_v p(v|C, d_{<i})$. Stop at the first mismatch.

Case A: every draft matches

Draft	a	red	car	
Target	a	red	car	p_4 adds "."
	match	match	match	

Commit C + a + red + car + .

Keep verified draft K/V. Process the bonus token next; synchronize the draft context.

One target pass commits 4 new tokens.

Case B: first mismatch at token 2

Draft	a	red	~~car~~	
Target	a	blue	car	
	match	reject	discard	

Commit C + a + blue

Keep C+a. Discard the red/car KV suffix. Feed "blue" next and restart the draft.

"car" used rejected context, so it cannot count.

- Commit only the accepted prefix; return the verified result so the draft resumes from the same context.
- Shown: exact greedy decoding. Sampling requires probability-based acceptance and corrected resampling.

Leviathan et al. (2023); Chen et al. (2023); Hugging Face, Assisted Generation (2023).

Lecture 2 recap

How each technique changes inference

Quantization	Fewer bits reduce storage and reads. Higher precision protects sensitive operations.
Prefill / decode / KV cache	Prefill processes the prompt. Decoding reuses earlier K/V at every layer and processes each new token.
MQA / GQA	Shared K/V heads reduce per-layer KV cache and cache traffic during decoding.
FlashAttention / PagedAttention	FlashAttention avoids large attention intermediates. PagedAttention manages persistent K/V in blocks.
MoE	Each token activates selected FFN experts. All expert weights still need storage.
Speculative decoding	A small model drafts. The target verifies, then accepts a prefix or corrects a rejected token.

A deployment decision you can defend

Question	Evidence to use
Can the workload fit?	Weight bytes plus KV and runtime memory
What is the bottleneck?	Prefill / decode timing and resource use
Which technique addresses it?	Precision, architecture, kernel or scheduling
Did the change help?	Quality, latency and useful throughput
Can others reproduce it?	Pinned model, runtime and test workload

Next lecture · training and fine-tuning the parameters whose execution we have now studied.